

# TTT와 Titans의 효율 — Scaling 병목과 해법, 그리고 memory device 집중 조명

neural-memory-study · 집중 조명 study paper · 근거등급 a=명시 · b=추론 · c=미명시

2026-07-16

## 차례

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 0. 들어가며 — 목적·범위·읽는 법                                            | 1  |
| Part I — 집중 조명                                                  | 2  |
| 1. TTT 집중 조명 — test-time에 학습되는 메모리 시스템                          | 2  |
| 2. Titans 집중 조명과 그 효율 전반                                        | 3  |
| Part II — Scaling 병목 3종                                         | 5  |
| 3. 병목 ① — model-size scaling (TTT/Titans)                       | 5  |
| 4. 병목 ② — training parallel/batch scaling (TTT/Titans)          | 6  |
| 5. 병목 ③ — serving batch inference + Prefill/Decode(P/D) scaling | 8  |
| Part III — 해법 3종 (무엇을·어떻게·왜 working)                            | 10 |
| 6. 해법 ① — model-size scaling 해결                                 | 10 |
| 7. 해법 ② — training parallel/batch scaling 해결                    | 12 |
| 8. 해법 ③ — serving/P/D batch inference 해결                        | 13 |
| Part IV — memory device 집중 조명                                   | 15 |
| 9. memory device 집중 조명 ① — 한계                                   | 15 |
| 10. memory device 집중 조명 ② — 개선으로 해결되는 해법                        | 16 |
| 11. 종합 — 세 병목 × 해법 × memory device 지도                           | 18 |
| 12. 열린 질문                                                       | 19 |
| References                                                      | 19 |
| 부록 A — 섹션별 검증 노트 (사실검증 요약)                                      | 21 |

## 0. 들어가며 — 목적·범위·읽는 법

이 글의 목적은 하나다. TTT(test-time training)와 Titans 계열을 '효율·scaling'의 시스템 관점에서 집중 조명하고, 세 가지 scaling 병목 — ① model-size, ② training의 parallel/batch, ③ serving의 batch inference(Prefill/Decode 포함) — 각각에 대해 (1) 병목이 왜 생기는지의 근거, (2) 그것을 푸는 논문들이 무엇을·어떻게·왜 working하는지, 그리고 (3) 이 모든 것이 만나는 memory device의 한계와 그 개선 해법을 정리한다.

세 가지 caveat. 1. 이 글은 pre-training 레시피나 아키텍처의 세부가 아니라 '효율·scaling 병목과 해법'의 시스템 관점이다. 같은 논문이라도 여기서는 효율 측면 조명한다. 2. 다루는 논문 상당수가 2025~2026년 것이라 수치·재현성이 미확정인 경우가 많다. 그래서 문장마다 근거등급을 달았고, 모든 수치는 best-effort 검증이다(9-agent 워크

플로: 7~10각도 심층 집필 → 적대적 사실검증). 3. **단일 논문으로는 그림이 불완전하다**. 병목과 해법, 이론과 시스템을 함께 봐야 이해된다. 서로 다른 갈래(distillation·병렬화·서빙)는 억지로 하나의 대서사로 통합하지 않았다.

**표기 주의 — LaCT 하드웨어**. LaCT[arXiv:2505.23884]의 “최대 ~70% GPU utilization”은 원문 보고 수치이나 실측 하드웨어(A100/H100) 표기가 소스마다 엇갈린다. 본 글은 **A100로 통일하되 하드웨어 세부**는 [b]로 둔다. abstract가 확정하는 것은 “hardware utilization을 orders-of-magnitude 개선”과 “nonlinear state를 모델 파라미터의 **최대 40%까지 확대**”이다 [a][arXiv:2505.23884].

**구조**. Part I(1-2장) TTT·Titans 집중 조명 → Part II(3-5장) 세 병목 → Part III(6-8장) 세 해법 → Part IV(9-10장) memory device 한계·개선 → 11장 종합 지도 → 12장 열린 질문 → References → 부록 A 섹션별 검증 노트.

## Part I — 집중 조명

### 1. TTT 집중 조명 — test-time에 학습되는 메모리 시스템

TTT란 무엇인가 — “은닉상태가 곧 학습 중인 모델”

Test-Time Training(TTT) 계열의 출발점은 하나의 재정의다. 통상적인 RNN에서 은닉상태  $h_t$ 는 과거를 압축한 고정 크기 벡터이고, 갱신은 학습된 게이팅/선형변환으로 이루어진다. TTT는 이 은닉상태를 **하나의 작은 모델의 가중치  $W_t$** 로 보고, 각 토큰마다 그 가중치를 **self-supervised gradient step**으로 갱신한다 [arXiv:2407.04620][a]. 구체적으로 입력 토큰  $x_t$ 로부터 학습된 투영으로 (key  $k_t$ , value  $v_t$ , query  $q_t$ )를 뽑아, “손상된 입력  $k_t$ 를 원래 값  $v_t$ 로 복원”하는 재구성 손실  $\ell(W; x_t) = \|f_W(k_t) - v_t\|^2$ 을 정의하고,  $W_t = W_{t-1} - \eta \nabla_W \ell(W_{t-1}; x_t)$ 로 한 스텝 내린 뒤 출력은  $z_t = f_{W_t}(q_t)$ 로 읽는다 [arXiv:2407.04620][a]. 즉 forward pass 안에 미니 학습 루프(inner loop)가 들어 있고, 은닉상태 = “지금 이 시퀀스에 대해 학습되고 있는 모델”이 된다 [arXiv:2407.04620][a]. TTT-Linear는  $f_W$ 가 선형모델, TTT-MLP는 2-layer MLP이며, 후자가 더 표현력 큰(비선형) 메모리를 갖는다 [arXiv:2407.04620][a]. 이 프레이밍의 실전 이점은 명확하다: Mamba 같은 고정 은닉상태 SSM은 16k 토큰 이후 더 이상 perplexity를 낮추지 못하는 반면, TTT 층은 문맥이 길어질수록 계속 손실을 줄인다 — 메모리 용량이 “벡터 크기”가 아니라 “모델 가중치 수”로 스케일하기 때문이다 [arXiv:2407.04620][a].

왜 이것이 새로운 것이 아니면서도 새로운가 — fast-weight 계보와 통일들

이 아이디어의 뿌리는 1990년대 Schmidhuber의 fast-weight programmer이며, Schlag et al.이 이를 현대적으로 정리했다: **linearized self-attention은 사실상 fast-weight programmer와 형식적으로 동치**라는 것이다 [arXiv:2102.11174][a]. 느린 네트워크(slow weights)가 key-value 외적  $k_t v_t^\top$ 을 순차적으로 누적해 빠른 메모리(fast weights)를 프로그래밍하고, query로 읽어낸다. Schlag의 핵심 관찰은 **순수 additive 외적 누적은 용량 한계 (key 충돌 시 덮어쓰기 불가)**를 갖는다는 점이고, 이를 **delta rule**(현재 매핑을 조회한 뒤 오차만큼만 정정)로 바꾸면 유한 메모리를 훨씬 효율적으로 쓴다는 것이다 [arXiv:2102.11174][a]. 여기서 delta rule은 바로 재구성 손실에 대한 gradient step과 같은 형태다 — TTT의 inner-loop이 사실상 이 delta-rule 계보의 일반화임을 보여준다 [arXiv:2102.11174][b].

이 관계를 가장 넓게 형식화한 것이 test-time regression 통일틀이다 [arXiv:2501.12352][a]. 여기서는 associative recall(문맥적 토큰 회수)을 **memorization=regression** 문제로 놓고, linear attention·SSM·fast-weight programmer·online learner·softmax attention을 모두 세 가지 설계선택 — (1) regression 가중치(토큰별 중 요도), (2) regressor 함수족(선형/MLP), (3) test-time 최적화 알고리즘(1-step GD, ridge closed-form 등) — 의 특수 사례로 유도한다 [arXiv:2501.12352][a]. 이 관점에서 TTT-Linear는 “선형 regressor를 SGD로 test-time에 fitting”하는 것에 지나지 않으며, 이는 왜 그렇게 많은 서로 다른 층들이 결국 비슷한 형태로 수렴하는지를 메커니즘적으로 설명한다 [arXiv:2501.12352][b].

이중성(duality) — parallel / recurrent / chunkwise, 그리고 dual form

효율의 열쇠는 이 계열이 **여러 계산 형태 사이의 수학적 동치**를 갖는다는 데 있다. Katharopoulos et al.은 softmax를 kernel feature map의 내적으로 근사하면 attention을  $O(N^2)$ 에서  $O(N)$ 으로 낮출 수 있고, 그 결과 **동일한 연산이 (a) 병렬 행렬곱 형태와 (b) 상태를 이월하는 recurrent 형태 두 가지로 표현됨**을 보였다 — autoregressive 추론에서 최대 4000배 가속 [arXiv:2006.16236][a]. RetNet은 여기에 세 번째 형태를 명시적으로 더한다: 학습에

유리한 **parallel**,  $O(1)$  추론에 유리한 **recurrent**, 그리고 긴 시퀀스를 위한 **chunkwise recurrent**(청크 내부는 병렬, 청크 간은 순차 요약) [arXiv:2307.08621][a]. 세 형태가 수치적으로 같은 출력을 내므로, 학습 때는 병렬로 GPU를 채우고 추론 때는 상태를 이월한다.

TTT 논문이 이 이중성을 실제 효율로 번역한 것이 **dual form**이다. inner-loop을 토큰마다 순차적으로(primal form) 돌리면 accelerator에서 참담하게 느리다 — TensorCore는 행렬-행렬 곱에 특화돼 있어, 벡터 단위의 순차 갱신으로는 유닛 대부분이 놀기 때문이다 [arXiv:2407.04620][a]. 대신 미니배치 내부의 gradient step들을 하나의 큰 행렬곱 묶음으로 재배열한 dual form은 출력이 primal과 동치이면서 논문의 JAX 구현 기준 primal 대비 5배 이상 빠르다 [arXiv:2407.04620][a]. 이것이 chunkwise의 TTT판이다: 청크(미니배치) 내부의 gradient들은 청크 시작점의 동일한  $W$ 에 대해 계산되어 병렬 행렬곱으로 묶이고,  $W$ 의 실제 이월은 청크 경계에서만 일어난다 [arXiv:2407.04620][b].

## 왜 순차성이 모든 병목의 뿌리인가

핵심 긴장은 여기 있다. TTT의 정의상  $W_t$ 는  $W_{t-1}$ 에 의존한다 — gradient step은 본질적으로 순차적이다 [arXiv:2407.04620][b]. dual form/chunkwise는 이 순차성을 완전히 없애지 못하고 청크 경계로 밀어낸다. 청크를 작게(예: 16-64 토큰) 잡으면 메모리 갱신은 세밀하지만 병렬화 단위가 작아 accelerator를 못 채운다. LaCT는 바로 이 지점을 정량화했다: 이런 소형 청크 TTT 층의 **FLOPs 이용률이 흔히 5% 미만**이라는 것이다 [arXiv:2505.23884][a]. 즉 순차 갱신이라는 정의가 하드웨어 활용률의 상한을 눌러버린다. LaCT의 처방은 정반대 — 청크를 2K~1M 토큰까지 **극단적으로 키워** 갱신을 compute-bound로 만들면 A100에서 이용률이 peak FLOPs의 70%를 넘는 수준까지 오른다 [arXiv:2505.23884][a]. 그러나 이는 공짜가 아니다: 큰 청크는 청크 내부에서 메모리 갱신을 지연시키므로 세밀한 순차 적응력과 하드웨어 효율 사이의 **명시적 trade-off**다 [arXiv:2505.23884][b].

또 하나의 균열은 “정말 meta-learning인가”라는 물음이다. 2026년 2월의 한 분석은 KV binding 기반 TTT의 online-meta-learning/memorization 해석과 모순되는 현상들을 지적하며, **복잡한 fast-weight(다층 MLP, momentum 포함) TTT조차 학습된 linear attention operator로 해석적으로 등가 재작성됨**을 증명한다 — inner loop은 진짜 학습이 아니라 query·key·value의 history-dependent 혼합을 유도할 뿐이라는 것이다 [arXiv:2602.21204][a]. 이 관점의 실익은 순차 inner-loop을 **완전 병렬 형태로 접고** 중복 요소(깊은 MLP·momentum)를 제거해, 복잡한 다층-MLP inner loop 아키텍처에서 최대 4.0배의 inference throughput을 얻는 것이다 [arXiv:2602.21204][a]. skeptic 관점에서 이 주장은 아직 신선하고(ICML 2026 채택), “모든 TTT  $\approx$  linear attention”이 어느 파라미터 영역·태스크까지 실질 성능 저하 없이 성립하는지는 논쟁 여지가 있다 [arXiv:2602.21204][c].

요컨대 TTT는 “test-time에 self-supervised gradient로 갱신되는 fast-weight 메모리”이며, 그 표현력의 원천인 **순차적 갱신**이 곧 이후 모든 시스템 병목(낮은 FLOPs 이용률, 청크 크기 딜레마, 병렬화 압박)의 근원이다. 이후 섹션들은 이 순차성을 어떻게 청크·dual form·linear-attention 등가로 우회하는가의 이야기이다.

## 2. Titans 집중 조명과 그 효율 전반

**Titans의 핵심 메커니즘: 왜 “test-time 학습”이 병목이자 해법인가** Titans의 출발점은 TTT 계열의 문제의식을 계승한다. 선형 RNN의 고정 크기 memory는 in-context retrieval에서 attention을 못 따라가는데, 그 근본 원인은 memory의 “쓰기 규칙이 너무 단순”하기 때문이다 [arXiv:2501.00663][b]. Titans는 long-term memory 모듈  $M$  자체를 **deep neural memory**(MLP; 실험에서 2층 이상의 deep 구조가 유리하다고 보고)로 두고, 각 토큰에서 이 MLP의 가중치를 test time에 갱신한다 [arXiv:2501.00663][a]. 갱신 신호는 associative memory loss  $\ell(M; x_t) = \|M(k_t) - v_t\|^2$ 의 gradient이며( $k_t, v_t$ 는  $x_t$ 의 선형 사영), 논문은 이 gradient를 “surprise”로 해석한다. 예측이 크게 빗나갈수록  $\nabla \ell$ 가 커지고 memory가 그만큼 크게 움직인다 — 놀라운 입력일수록 강하게 각인된다는 직관을 수식화한 것이다 [arXiv:2501.00663][b].

여기에 두 가지 장치가 더해진다. 첫째, **momentum**:  $S_t = \eta_t S_{t-1} - \theta_t \nabla \ell$ 로 과거 surprise를 누적한다. 순간 gradient만 쓰면 놀라움 이후 이어지는 관련 토큰들을 놓치는데, momentum이 “놀란 사건 주변의 흐름”까지 기억하게 만든다 [arXiv:2501.00663][b]. 둘째, **weight decay**  $\alpha_t$ 를 forgetting gate로 재해석한다:  $M_t = (1 - \alpha_t)M_{t-1} + S_t$ .  $\alpha_t$ 가 데이터 의존적이므로, memory가 가득 찰 때 무엇을 지울지를 입력에 따라 조절한다 [arXiv:2501.00663][a]. 구조적으로 이 갱신식은 **weight decay가 붙은 momentum SGD** 그 자체이며 — 논문 스스로도 이 대응을 명시한다 — “memory = 내부 optimizer의 상태”라는 이후 Nested Learning/HOPE의 관점이 이미 여기 배태되어 있다 [arXiv:2501.00663][b]. Ablation에서 convolution, momentum, weight decay, persistent memory가 각각 성능에 기여한다고 보고된다 [arXiv:2501.00663][a].

memory 분기를 short-term attention과 결합하는 방식이 세 변주다. **MAC**(Memory as Context)는 retrieve한 memory를 attention의 추가 context로 앞에 붙이고, **MAG**(Memory as Gate)는 memory 출력과 sliding-window attention을 gating으로 합치며, **MAL**(Memory as Layer)는 memory를 attention 앞단의 한 층으로 쌓는다 [arXiv:2501.00663][a].

**효율의 실체: chunk-parallel과 deep memory의 이중 비용** Titans의 효율은 이율배반적이다. 이론상 memory 갱신은 토큰마다 순차적이라 GPU 병렬화에 최악이다. 이를 살리는 것이 **chunk-parallel** 학습이다. 시퀀스를 크기  $b$  청크로 자르고, 청크 내부의 gradient 갱신을 mini-batch gradient descent로 근사해 matmul로 tensorize한다(TTT-DeltaNet 계열의 공통 트릭) [arXiv:2501.00663][b]. 핵심 트레이드오프는 청크 크기  $b$ 다:  $b$ 가 크면 하드웨어 이용률이 올라 빠르지만, 청크 내 순차 recurrence를 거칠게 근사하므로 정확도가 떨어진다. 즉 속도와 정확도가 하나의 하이퍼파라미터에 묶여 **고정된 타협**을 강요당한다 — 이 지점이 뒤의 TNT가 정면으로 겨냥하는 병목이다 [arXiv:2511.07343][a].

두 번째 비용은 memory가 단일 행렬이 아니라 **deep MLP**라는 데서 온다. 토큰(또는 청크)마다 memory MLP의 forward와 backward를 모두 통과해야 surprise gradient가 나오므로, 층 수에 비례하는 연산이 linear attention 대비 그대로 얹힌다 [b]. 표현력(비선형 연상)을 사는 대가가 곧 forward/backward 왕복 비용이며, Titans 계열이 “느린 학습·낮은 이용률”이라는 평판을 얻은 실질적 이유다 [arXiv:2511.07343][a].

**계보의 확장: Atlas, TNT, HOPE, Sleep** Atlas는 Titans 갱신 규칙의 표현력을 끌어올린다. 문제의식은 memory가 현재 토큰만 최적화하는 online 방식이라 과거 문맥에 대한 최적성을 잃는다는 것이다 [arXiv:2505.23735][a]. 해법인 **Omega rule**은 delta rule을 슬라이딩 윈도우/문맥 전체 기억으로 일반화해 현재뿐 아니라 과거 토큰들에 대해서도 memory를 동시에 최적화하고, key/query에 polynomial feature mapping을 얹어 용량을 키운다 [arXiv:2505.23735][a]. 나아가 내부 갱신에 **Muon optimizer**(Newton-Schulz 직교화 기반의 근사 2차 정보)를 도입해 gradient를 그대로 쓰지 않고 locally optimal한 갱신을 얻는다 [arXiv:2505.23735][a]. 비용은 2차 정보(또는 그 근사)를 매 갱신마다 계산하는 것이지만, Atlas는 BABILong의 10M context length에서 80%+ 정확도를 달성하며 Titans의 장문 성능을 개선했다고 보고한다 [arXiv:2505.23735][a].

**TNT**는 앞서 짚은 청크 크기 딜레마를 학습·추론 분리로 푼다. 1단계는 큰(하드웨어 친화적) 청크로 도는 **hierarchical memory** 사전학습으로, global 모듈이 큰 청크로 장거리 문맥을, 병렬 local 모듈들이 세부를 맡으며, local memory state를 주기적으로 리셋해 순차 의존을 끊고 대규모 context 병렬화를 가능케 한다. 2단계는 local 모듈만 작은 고해상도 청크로 짧게 fine-tune해 정확도를 회복한다 [arXiv:2511.07343][a]. 결과적으로 가장 정확한 baseline 구성 대비 최대 **17× 빠른 학습**과 동시 정확도 향상을 보고한다 — 속도를 정확도와 탈동조화(decouple)한 것이 핵심 기여다 [arXiv:2511.07343][a].

**Nested Learning / HOPE**는 한 걸음 더 추상화해, 아키텍처와 optimizer를 서로 다른 주파수에서 도는 중첩·다층 최적화 문제들로 재정의한다(“The Illusion of Deep Learning Architectures”) [arXiv:2512.24695][a]. 세 축은 (i) SGD-momentum-Adam 같은 optimizer 자체를 gradient 정보를 압축하는 associative memory로 보는 관점, (ii) 자신의 갱신 규칙(update algorithm)을 스스로 학습하는 **Self-Modifying** 시퀀스 모델, (iii) 장·단기 memory 구분을 빈도 스펙트럼으로 일반화한 **Continuum Memory System(CMS)**다 [arXiv:2512.24695][a]. 효율 관점에서 CMS는 “빠른 모듈은 자주, 느린 모듈은 드물게” 갱신하도록 빈도를 계층화해 갱신 비용을 분산하는 장치로 읽을 수 있다 [b]. 이를 결합한 HOPE(Titans의 한 변형)는 language modeling·knowledge incorporation·few-shot generalization·continual learning·long-context reasoning에서 promising results를 보고한다 [arXiv:2512.24695][a].

**Sleep**(“Language Models Need Sleep”)은 online 학습 비용과 consolidation 비용을 시간축에서 분리한다 [b]. 짧고 불안정한 short-term memory를 안정적 장기 지식으로 이관하는 두 단계 — Memory Consolidation과 Dreaming — 를 두는데, Consolidation의 **Knowledge Seeding**은 작은 self의 memory를 큰 네트워크로 상향(upward) distill해 용량을 늘리면서 지식을 보존하고, Dreaming은 RL로 합성 커리큘럼을 생성해 새 지식을 리허설한다 [arXiv:2606.03979][a]. 무거운 consolidation을 추론 경로에서 빼 offline으로 미루는 발상이지만, sleep 스케줄 비용과 forgetting 방지의 실제 균형은 아직 폭넓게 검증된 값이 부족하다 [c].

**Skeptic: chunking이 baseline을 못 넘길 때** Titans Revisited는 원 논문의 코드 부재와 서술 모호성으로 재현이 어려웠던 상황에서 경량 재구현을 내놓고 Masked Language Modeling·시계열 예측·추천 과제로 검증한다 [arXiv:2510.09551][a]. 결론은 균형 잡혀 있다: chunking 때문에 Titans가 확립된 baseline을 **항상 넘지는 못하지만**, Neural Memory 구성요소 자체는 attention-only 대비 일관되게 성능을 올린다 [arXiv:2510.09551][a]. 이는 앞선 효율 논의와 정확히 맞물린다 — 속도를 위한 chunking 근사가 Titans의 실측 이득을 잡아먹을 수 있으며,

원 논문의 화려한 수치를 조건 없이 받아들이면 안 된다는 신호다 [b]. 정리하면, Titans 계보의 효율 서사는 “deep memory의 표현력 vs. 순차 갱신의 병렬화 비용”이라는 한 축의 반복적 재협상이고, Atlas는 표현력, TNT는 학습 속도, HOPE·Sleep은 갱신 빈도/시점의 분리로 각자 다른 지점을 공략한다 [b].

**Part I 요약.** TTT/Titans는 ‘추론 중 self-supervised gradient로 fast weight를 갱신’해 표현력(문맥 기억)을 얻는다. 그러나 그 갱신은 **본질적으로 순차적**이며, 바로 이 순차성이 이후 세 병목(낮은 GPU util·병렬화 압박·배치 불가)의 공통 뿌리다.

## Part II — Scaling 병목 3종

### 3. 병목 ① — model-size scaling (TTT/Titans)

**병목의 출발점:** TTT/Titans는 “파라미터”가 아니라 “메모리 용량”으로 문맥을 저장한다

TTT와 Titans 계열의 핵심 아이디어는 문맥을 KV 캐시가 아니라 test-time에 gradient로 갱신되는 내부 상태 (matrix 또는 MLP 형태의 neural memory)에 압축 저장하는 것이다 [arXiv:2501.00663]. 따라서 “모델을 키운다”는 것은 두 갈래로 나뉜다 — (i) 백본 layer/width를 키우기, (ii) neural memory 모듈 자체를 키우기. 병목은 주로 (ii)에서 나온다. 문맥 저장은 본질적으로 associative memory 문제이고, associative memory의 용량은 **파라미터 수에 선형으로 비례하지 않기** 때문이다 [b].

**용량 이론:  $d^2$  파라미터에 왜  $\sim d$  정보만 담기는가**

Atlas는 이 병목을 정면으로 정식화한다. 단순 matrix memory  $M \in \mathbb{R}^{d_v \times d_k}$ 의 용량은 Proposition 1에서 “선형 독립 key 기준 최대  $\mathcal{O}(d_k)$ 개의  $(k_i, v_i)$  쌍”으로 제한되며, 이는 파라미터 수  $d_v d_k$ 에 대해 **sub-linear**(고정 크기 메모리  $M$ 이 담는 독립 패턴 수가  $c M$ 보다 엄격히 작다)라고 명시한다 [a][arXiv:2505.23735]. 즉  $d^2$ 개의 파라미터를 쥐고도 신뢰성 있게 회수 가능한 정보는  $\sim d$  수준에 그친다. 이는 고전 결과와 정합적이다 — Hopfield associative memory의 저장 용량은  $p \approx 0.138 N$  패턴에 불과하고 (Amit-Gutfreund-Sompolinsky, 1985) [a], 선형 associative memory의 통계적 저장 문턱은 임계 패턴 수  $p_c$ 에 대해  $p_c \log p_c / d^2 = 1/2$ , 즉  $d^2 \approx 2 p_c \log p_c$ 로 정확히 특성화된다 — 패턴 수를 늘리려면 파라미터가  $\sim n \log n$ 처럼 필요하다는 뜻이다 [a][arXiv:2605.10795]. 메커니즘적으로, matrix memory는 outer-product 함으로 patterns를 겹쳐 쓰는데, 키들이 직교하지 않는 순간 cross-talk(간섭)이 회수 오차로 나타나고, 이 간섭이 회수 가능한 패턴 수를  $\text{rank} \approx d_k$ 로 상한 지운다 [b].

Atlas가 제시하는 탈출로가 오히려 병목의 구조를 드러낸다. 깊은 MLP memory(2층 이상)는 용량을 최소  $\mathcal{O}(d_k d_v)$ 까지 올리지만 상한이 여전히 key/value 차원에 대해 **subquadratic**이고 (Theorem 1) [a][arXiv:2505.23735], degree- $p$  polynomial feature map  $\phi_p$ 를 쓰면 용량이  $\mathcal{O}(d_k^p)$ 로 초선형화된다 (Proposition 2) [a] — 이  $d_k^p$ 는 degree- $p$  단항식 개수  $\binom{d_k+p}{p}$ 에서 나오는 실효 차원 확장으로 이해된다 [b]. 다시 말해 **용량을 늘리는 지렛대는 파라미터 수가 아니라 feature 차원/비선형성**이라는 것이 요점이다. 단순히 memory MLP의 width를 키우는 것으로는 용량이 파라미터에 비례해 늘지 않고, 저장 규칙(feature map, depth)을 바꿔야만 한다 [b]. Atlas가 online(마지막 토큰만 최적화, 즉 window  $c = 1$ ) 대신 문맥 window 전체를 최적화하는 Omega Rule( $\sum_{i=t-c+1}^t \gamma_i^{(t)} \|\mathcal{M}(k_i) - v_i\|_2^2$  최소화)을 도입한 것도, 고정 용량 안에서 무엇을 저장할지의 **배분** 문제가 규모의 진짜 병목임을 시사한다 [a].

한 가지 덜 알려진 축: 같은 용량이라도 **어떤 옵티마이저로 채우느냐**가 회수 품질을 좌우한다. Muon이 Adam보다 빠른 이유가 LLM의 associative-memory 파라미터(Value-Output(VO) attention weight와 FFN)에 있으며, Muon의 갱신이 linear associative memory의 outer-product 구조와 정렬되어(더 isotropic한 singular spectrum) heavy-tailed 분포의 tail class를 더 균형 있게 학습한다는 분석이 있다 [a][arXiv:2509.26030]. 다만 이 논문 자체는 사전학습 옵티마이저를 다루며, test-time 갱신 규칙(사실상 내부 옵티마이저)이 유한 용량의 실효 활용률을 결정한다는 TTT 해석은 본 저술의 외삽임을 밝혀 둔다 [b].

**Chunk mismatch: 규모에서 열화하는 실증**

Titans Revisited는 재현 실험에서 chunk가 규모/과제에 따라 병목이 됨을 구체적으로 보인다. chunk size는 통상 32-128 토큰이며(기본값 32) “chunk가 클수록 성능이 좋지만 계산비용이 커진다”는 accuracy-efficiency

tradeoff가 명시적으로 관찰된다 [a][arXiv:2510.09551]. recommendation(MovieLens 1M)에서 Titans MAC은 memory를 넣어도(MRR 0.3418→0.4371) BERT4Rec baseline(MRR 0.4451)을 끝내 넘지 못했고, 저자는 그 원인을 chunking으로 지목하되 “memory 성분이 chunking으로 인한 정보 손실을 완화한다”고 명시한다 [a]. 메커니즘은 이렇다: chunk 단위로 memory를 갱신하면 chunk 내부는 병렬 attention, chunk 경계에서만 상태가 넘어가므로, 문맥이 커져 chunk 수가 늘면 경계에서의 정보 압축 손실이 누적되고 fixed-capacity 상태가 포화한다 [b]. 여기에 원논문의 코드 부재·기술 모호성으로 재현성 자체가 논란이라는 점(저자들이 명시)은 skeptic 관점에서 반드시 병기해야 한다 [a][arXiv:2510.09551].

## 고정 용량 recurrent state의 정보이론적 벽

파라미터를 아무리 키워도 못 넘는 벽이 있다. Impossibility Triangle은 Efficiency(스텝 계산이 문맥 길이와 무관)·Compactness(상태 크기가 문맥 길이와 무관)·Recall(문맥 길이에 비례하는 사실 회수)의 셋 중 최대 둘만 가능함을 Data Processing Inequality와 Fano 부등식으로 증명한다. Efficiency+Compactness를 만족하면 회수 가능한 KV 쌍은 **문맥 길이와 무관하게**  $\mathcal{O}(\text{poly}(d)/\log V)$ 로 상한된다 [a][arXiv:2605.05066]. TTT/Titans는 정의상 compact한 고정 상태를 쓰므로 이 삼각형의 recall 꼭짓점을 구조적으로 포기한 셈이고, 긴 문맥에서의 열화는 모델 크기  $d$ 의 다항식 함수로만 완화될 뿐 근본적으로 제거되지 않는다 [b]. Conformal-sympow도 동일 병목을 실증한다 — “recurrent state의 유한 용량이 정보 보존을 제한해, 학습/평가 문맥 길이를 키우면 성능이 열화”하며, 이를 data-dependent multiplicative gating으로 용량을 동적으로 비워야(+ data-dependent rotary embedding) 완화된다 [a][arXiv:2503.03269].

## Hybrid 비율 의존과 scaling-law tradeoff

그래서 실무 규모화는 순수 recurrent가 아니라 full attention을 섞는 hybrid로 간다. 72개 모델(340M 36개·1.3B 36개)의 체계 연구는 “language modeling loss는 linear:full 비율에 대체로 둔감하지만 recall은 full attention 비중이 클수록 크게 개선”됨을 보이고, Transformer급 recall에는 linear:full = 3:1-6:1 비율을 권한다 [a][arXiv:2507.06457]. 이는 규모화의 이득이 memory 파라미터가 아니라 **소수의 full-attention layer** 예산에 걸려 있다는 뜻으로, TTT/Titans를 그냥 파라미터로 키우는 것이 왜 안 통하는지의 실증이다 [b].

마지막으로, 고정 예산을 어느 축에 쓸지의 tradeoff는 scaling law로 정량화된다 — 다만 이 연구(ELM Network)는 생물학적 cortical neuron에서 영감을 받은 일반 recurrent net을 SHD-Adding·Enwik8에서 평가한 것으로, TTT/Titans에 직접 적용된 결과가 아니라 **유비(analogy)**로 읽어야 한다 [c]. 예산  $P$ 를 뉴런 수  $N$ ·per-unit 실효 복잡도  $k_e$ ·연결성  $k_c$ 로 쪼갤 때,  $k_e$ 를 키우면 per-channel 신호는 강해지지만 채널 수  $N$ 이 줄어 고정 예산 아래 non-trivial한 최적점이 생기며, 예산이 커질수록 더 많고 더 복잡한 뉴런 쪽으로 최적점이 이동한다 [a][arXiv:2605.12049]. 저자들의 정보이론 모델은 양극단의 diminishing return을 **per-neuron SNR 포화**와 **across-neuron redundancy** 두 기제로 귀속시킨다 [a]. (원문에 대응하는 구체적 노이즈 감쇠 형태나  $d_m \sim \sqrt{N_{rec}}$  류의 스케일링은 본 저술이 확인하지 못했으므로 그대로 인용하지 않는다 [c].) 요약하면 **단일 축(memory width) 확대는 용량 sub-linearity·삼각형 recall 벽·hybrid 비율 의존이 겹쳐 diminishing return에 빠지고, 예산은 저장 규칙·feature 차원·full-attention layer 같은 다른 축으로 배분해야** 한다는 것이 model-size scaling 병목의 통합 메커니즘이다 [b].

## 4. 병목 ② — training parallel/batch scaling (TTT/Titans)

### 병목의 뿌리: inner-loop의 순차 의존성

TTT/Titans 계열의 정의적 특징은 시퀀스를 흐르며 “빠른 가중치(fast weight)” 상태  $W_t$ 를 self-supervised 손실의 gradient로 갱신한다는 점이다. TTT는 이 갱신을 명시적으로 “hidden state를 online gradient descent로 학습하는 것”으로 정식화하고, Titans의 neural memory도 momentum과 weight-decay(forgetting)를 갖는 gradient 갱신  $W_t \leftarrow W_{t-1} - \eta_t \nabla \ell(W_{t-1}; x_t)$  형태를 쓴다(위 식은 momentum 항을 생략한 도식적 표기) [a][arXiv:2407.04620][arXiv:2501.00663]. 문제는  $W_t$ 가  $W_{t-1}$ 에 의존하고, 그 의존이 **비선형**(gradient가  $W_{t-1}$ 의 비선형 함수)이라는 데 있다. 선형 attention/SSM의 상태 재귀  $S_t = a_t S_{t-1} + b_t$ 는 결합법칙이 성립하는 스캔(prefix scan)으로  $O(\log T)$  깊이로 병렬화되지만, gradient 갱신은 그런 대수적 구조를 갖지 않아 원리적으로 시퀀스 축에서 곧장 병렬화되지 않는다 [b]. 이것이 “학습이 prohibitively slow하고 hardware utilization이 낮다”는 TNT의 진단의 근원이다 [a][arXiv:2511.07343].

## 왜 GPU util이 낮은가: chunk 크기의 정량적 tradeoff

병렬화의 표준 우회로는 시퀀스를 chunk로 쪼개 chunk 내부에서만 갱신을 근사·병렬화하는 것이다. Titans는 memory를 매 토큰이 아니라 **직전 chunk의 끝 시점** 상태를 기준으로 갱신함으로써 chunk 내부의 모든 갱신을 동시에 계산하고, learning rate·weight decay·momentum을 matmul과 함으로 재작성해 GPU의 matmul 유닛을 태운다 [a][arXiv:2501.00663]. TTT도 같은 동기에서 mini-batch TTT와 dual form을 도입해 mini-batch 내부를 matmul로 처리한다 — “matmul이 충분치 않으면 GPU가 논다”는 것이 명시적 이유다 [a][arXiv:2407.04620].

그러나 chunk 크기  $b$ 는 양날의 검이다. LaCT는 이를 가장 선명하게 정량화한다: 기존 TTT류가 16~64 토큰마다 fast weight를 갱신하는 **작은** online minibatch를 쓰기 때문에 FLOPs utilization이 종종 5% **미만**에 그치고, chunk를 2K~1M 토큰으로 키우면 연산이 compute-bound로 바뀌어 **NVIDIA A100에서 GPU utilization을 최대 70%까지 끌어올린다고 보고한다** [a][arXiv:2505.23884]. 메커니즘은 명확하다 — 작은 chunk는 갱신마다 상태-크기 행렬을 읽고 쓰는 I/O가 지배해 memory-bound가 되고, 산술강도(arithmetic intensity)가 낮아 텐서코어가 대부분 유휴 상태가 된다 [b]. 큰 chunk는 하나의 큰 matmul로 묶여 산술강도가 올라간다 [b].

문제는 큰 chunk가 **표현력과 의존성**을 희생한다는 점이다. chunk 내부를 “**직전 chunk 끝 상태 기준**”으로 근사한다는 것은, chunk 길이만큼 상태 갱신을 **얼리는(freeze)** 것과 같아 chunk 내부의 미세한 토큰-단위 memorization이 사라진다 [b]. TNT는 이를 chunksize가 지배하는 상충으로 명시하며 — “큰 chunk는 속도를 올리지만 성능을 떨어뜨려, 고정된 suboptimal 타협을 강요한다”고 진단한다 [a][arXiv:2511.07343]. TNT의 해법 자체가 이 상충의 존재를 증언한다: (1) global module가 크고 hardware-friendly한 chunk로 장거리 문맥을 처리하고 여러 local module가 세밀한 부분을 병렬로 담당하되 **주기적으로 local memory state를 리셋해** 순차 의존을 끊음으로써 대규모 context parallelization을 얻는 pre-training 단계, (2) local module만 작은 고해상도 chunk로 짧게 fine-tune하는 단계로 분리해, **가장 정확한 baseline 설정 대비 최대 17배 빠른 학습**을 달성했다고 보고한다 [a][arXiv:2511.07343]. 즉 “속도(큰 chunk) vs 정확도(작은 chunk)”를 학습 단계 분리로 우회한 것이며, 단일 chunk 크기로는 두 목표를 동시에 만족할 수 없다는 병목을 역으로 확인해준다 [b].

## backprop-through-the-inner-loop의 비용

학습 시에는 outer loop이 inner loop의 갱신 궤적을 통해 gradient를 역전파해야 한다. inner 스텝이  $T/b$ 개면 원칙적으로 그만큼의 중간 fast-weight 상태를 activation으로 보존해야 하므로, 메모리가 inner 스텝 수에 비례해 증가하는 unrolled-optimization 특유의 비용이 생긴다 [b]. Titans/TTT가 chunk-wise 병렬 형태를 굳이 matmul-only로 재유도하는 이유 중 하나가 이것이다 — 상태를 매 토큰 materialize하지 않고 chunk 경계에서만 다루면 저장할 중간 상태가 줄어든다 [b]. DeltaNet 병렬화(WY representation)가 이 문제를 가장 깔끔하게 보여 준다: DeltaNet은 delta rule로 associative recall을 강화하지만 기존 학습 알고리즘이 시퀀스 축으로 병렬화되지 않아 학습이 비효율적이었는데, 갱신을 일반화된 Householder 변환의 곱으로 재파라미터화하고 compact한 WY 표현을 쓰면 **시점마다 행렬 크기의 hidden state를 materialize하지 않고도** chunkwise 병렬 전략을 확장할 수 있어, 1.3B 모델을 100B 토큰까지 올려 Mamba·GLA 대비 perplexity·zero-shot 성능에서 앞섰다 [a][arXiv:2406.06484]. 핵심은 “무엇을 저장하지 않아도 되게 만드느냐”가 메모리·I/O 병목을 좌우한다는 점이다 [b].

## batch × state footprint와 sequence-parallelism 난점

이 계열의 상태는 벡터가 아니라 **행렬(또는 작은 MLP의 가중치)**이다. LaCT는 nonlinear state size를 모델 파라미터의 **최대 40%까지** 키워 상태 용량(capacity)을 늘리는 것을 이점으로 내세우는데 [a][arXiv:2505.23884], 이는 곧 batch의 각 시퀀스가 이 큰 상태를 하나씩 들고 있어야 함을 뜻한다. 따라서 **활성 메모리  $\approx$  batch × state footprint**로 늘어나, 상태 용량을 키울수록 batch 확장 여지가 줄어드는 직접적 상충이 생긴다 [b]. 또한 Transformer 학습의 표준 무기인 sequence parallelism이 여기서는 곧장 통하지 않는다 — attention은 위치 간 결합이 대칭적이라 시퀀스 축을 device로 쪼개기 쉽지만, inner-loop 갱신은 chunk  $k$ 가 chunk  $k-1$ 의 최종 상태를 입력으로 요구하는 **좌→우 강결합**이라 쪼개 device 간에 상태를 순차 전달해야 한다 [b]. TNT가 “local state를 주기적으로 리셋해 순차 의존을 끊음으로써 context parallelization을 가능케 한다”고 굳이 설계한 것이, 리셋 없는 sequence-parallel이 어렵다는 방증이다 [b][arXiv:2511.07343].

## 비선형 recurrence의 병렬화: 가능하지만 조건부

“비선형 재귀는 병렬화 불가”는 절대적 명제가 아니라 **조건부**다. DEER는 비선형 재귀를 고정점 문제로 재서술해 Newton 방법(quadratic convergence)으로 시퀀스 축을 병렬 반복하며, **평가에서 최대 3자릿수(orders of magnitude), 학습에서 10배 이상**의 가속을 출력 변화 없이 얻었다고 보고한다 [a][arXiv:2309.12252]. ParaRNN

은 이를 LLM 규모로 밀어붙여, 비선형 재귀 전체를 하나의 방정식계로 보고 Markovian 구조에서 나오는 **block bi-diagonal** Jacobian에 맞춘 custom parallel reduction으로 Newton을 풀어 naive sequential 대비 **최대 665배** 가속과 7B 모델 학습(Transformer/Mamba2급 perplexity)을 달성했다 [a][arXiv:2510.21450]. 다만 이 접근들은 공짜가 아니다. Newton 반복은 Jacobian 관련 상태를 들고 있어야 해 메모리가 무겁고, 무엇보다 수렴이 **시스템의 예측가능성(predictability)**에 좌우된다: Gonzalez·Kozachkov 등은 dynamics와 최적화 문제의 조건 수(PL constant) 사이의 정확한 관계를 세우고, 예측가능성을 largest Lyapunov exponent로 정량화해, **예측 가능한 시스템에선 상태 궤적을 최악  $O((\log T)^2)$  시간에 병렬 계산할 수 있어 sequential 대비 큰 개선을 얻지만 혼돈적(chaotic)·예측불가 시스템에선 조건수가 나빠 parallel 수렴이 너무 느려 실용성이 없다**는 것을 보였다 [a][arXiv:2508.16817]. 나아가 순진한 DEER는 수치적으로 불안정해, quasi-Newton 근사(quasi-DEER)와 Levenberg-Marquardt-Kalman smoothing 연결을 이용한 안정화(ELK)가 필요하다는 후속 연구가 있다 [a][arXiv:2407.19115]. 요컨대 병렬 solver는 병목을 “없애는” 것이 아니라 “메모리·수렴안정성·데이터 예측가능성”이라는 새 축으로 **옹기는** 것이다 [b].

**skeptic 주석.** 이들 가속 수치는 서로 다른 baseline·정밀도·하드웨어에서 측정된 것이라 직접 비교는 위험하다 — LaCT의 “최대 70% GPU util”은 A100·특정 modality 기준 [a][arXiv:2505.23884], ParaRNN의 665 배는 naive sequential 대비 [a][arXiv:2510.21450], DEER의 “10배”는 특정 RNN/NeuralODE 학습 대상 [a][arXiv:2309.12252]이다. 특히 DEER류의 “출력 변화 없이”는 예측가능 영역에서의 주장이며, chaotic 영역에선 parallel 수렴이 무용할 만큼 느려진다는 점이 이론적으로 명시돼 있다 [a][arXiv:2508.16817]. 따라서 “비선형 recurrence도 병렬 학습 가능”은 성립하되, TTT/Titans의 fast-weight 갱신이 어느 예측가능성 영역에 있는지가 실제 이득을 좌우하며 이 부분은 아직 계열별로 충분히 규명되지 않았다 [c].

## 5. 병목 ③ — serving batch inference + Prefill/Decode(P/D) scaling

**병목의 근원 — batching이 전제하는 “공유 static weight”가 깨진다**

표준 LLM 서빙의 throughput은 batching에서 나온다. 서로 다른 request들이 **동일한 static weight  $W$** 를 공유하기 때문에, 여러 시퀀스의 토큰을 한 tensor로 쌓아 하나의 GEMM으로 흘려보내면 weight를 한 번 읽어 여러 request의 연산을 상각(amortize)할 수 있다. decode가 memory-bandwidth bound인 이유도 여기 있다 — 토큰당 연산량은 작는데 매 스텝 거대한  $W$ 를 HBM에서 다시 읽어야 하므로, batch size를 키워 그 읽기를 나눠 갖는 것이 유일한 탈출구다 [b]. PagedAttention/vLLM은 이 위에서 KV cache를 페이지 단위로 관리해 batch 점유율을 끌어올린다 [arXiv:2309.06180][a].

TTT·fast-weight 계열은 바로 이 전제를 정면으로 깬다. In-Place TTT는 MLP block의 최종 projection 행렬 자체를 **fast weight로 지정해 inference 중에 in-place로 갱신**한다(입력 projection은 frozen slow weight로 두고, NTP-aligned objective + chunk-wise update 사용) [arXiv:2604.06169][a]. 즉 request마다 자기만의 mutable weight/state를 소유한다. 이러면 “하나의  $W$ 를 공유”라는 batching의 대전제가 성립하지 않는다. request A와 B의 fast weight가 다르므로 두 request의 decode를 순진하게 한 GEMM에 묶으면 서로의 state를 오염시킨다 [b]. RW-TTT는 이 문제를 명시적으로 정식화한다: serial 실행은 정확하지만 느리고, naive batching은 request state를 corrupt시킨다 [arXiv:2605.28053][a]. 해법은 각 decode 스텝에 **owner·version·READ/WRITE effect** 태그를 붙여 “호환 가능한 phase만” 함께 batch하고, 갱신은 오직 소유자에게만 commit하는 것이다 [a]. 이 규율이 왜 working하는가: READ-only 국면(예: fast weight를 읽어 forward만 하는 chunk)은 공유  $W$ 가 여전히 성립하므로 batch가 안전하고, WRITE 국면만 소유자별로 격리하면 정확성과 병렬성을 동시에 회수할 수 있기 때문이다 [b]. 실측으로 GPU 한 장·8개 In-Place-TTT fast-weight stream에서 aggregate 274.61 tok/s로, sequential 대비 9.31×, 동일 메모리 예산의 per-stream replica 대비 3.44× 빠르며 RULER long-context 벤치마크의 동작을 보존한다 [arXiv:2605.28053][a]. 뒤집어 말하면, 특수 스케줄러 없이는 fast-weight 서빙이 sequential에 가깝게 붕괴한다는 뜻이다 [b].

**recurrent state는 KV처럼 prefix-cache되지 않는다**

일반 LLM에서 multi-turn·RAG·system-prompt 공유는 **prefix caching**으로 크게 절약된다. KV cache는 토큰별로 분해되는 append-only 자료구조라, 공유 prefix의 KV를 그대로 재사용하고 뒤 토큰만 이어 붙이면 된다 [b]. recurrent/linear-attention/TTT의 state는 이 성질이 없다. hidden state  $S_t$ 는 prefix 전체를 하나의 고정 크기 텐서로 **압축·덮어쓰기(overwrite)**한 결과라, “prefix의 처음  $k$  토큰까지의 상태”를 토큰 단위로 잘라내거나 부분 재사용할 수 없다 [b]. Marconi가 지적하듯, recurrent layer의 in-place state update는 부분 overlap에 대한 cache entry roll-back을 원천적으로 막아 exact-match hit만 허용하게 만들고, 이 때문에 prefix caching 같은 보완 최적화의 적용이 복잡해진다 [arXiv:2411.19379][a]. Marconi는 recency뿐 아니라 재



사용 예측과 FLOP 대비 메모리 footprint를 함께 보는 admission/eviction 정책으로 이를 우회해 token hit rate 최대  $34.4\times$ , TTFT 최대 71.1%(약 617 ms) 감소를 얻는다 [a] — 바꿔 말해 hybrid에서는 무엇을 캐싱할지가 KV보다 훨씬 비자명하다는 방증이다 [b]. Sparse Prefix Caching은 다른 각도로 접근한다: recurrent state를 **sparse한 checkpoint 위치에서만 정확히 저장**하고, cache hit 시 가장 깊은 checkpoint에서 resume해 남은 suffix를 정확 재계산한다. checkpoint 배치를 overlap-depth 분포 위의 배치 문제로 정식화해 exact  $O(NM)$  DP로 푼다 [arXiv:2605.05219][a]. 핵심은 SSM state가 하나의 텐서로 정확히 추출·복원 가능하다는 점을 이용하되, 임의 위치 재사용은 불가하므로 “몇 군데 스냅샷 + suffix 재계산”으로 타협한다는 것이다 [b].

HYPIC는 여기서 한 걸음 더 나아가, position-independent caching(비연속 segment의 KV 재사용)과 hybrid-attention이 **기존 시스템에서 공존하지 못하는 이유**를 정면으로 짚는다: per-token KV 재사용 primitive가 per-request recurrent state로 전이되지 않기 때문이다 [arXiv:2607.01299][a]. 해법은 linear-attention 층에 대해 **segment-cumulative transition operator**라는 대수적 primitive를 찾아, 각 segment의 zero-start end-state와 함께 캐싱해 독립 segment의 state를 상수 시간에 near-exact하게 합성하는 것이다 [a]. full-attention 층은 segment 경계의 작은 seam window만 재계산해 cross-segment lookback을 복원한다 [a]. 이 “선형 층은 합성 가능, full 층은 seam만 재계산”이라는 비대칭 처리가 hybrid 서빙 설계의 본질을 드러낸다 — 4개 hybrid 모델·5개 workload에서 동일 SLO 대비 TTFT를 최대  $2.45\times$  낮추고 peak throughput을 최대  $2.0\times$  높인다 [a].

### prefill(흡수) vs decode(순차 갱신): state-bandwidth-bound라는 새 축

일반 LLM에서 prefill은 compute-bound(프롬프트 전체를 병렬 흡수해 KV 구축), decode는 KV 읽기로 인한 memory-bandwidth-bound다 [arXiv:2311.18677][a]. recurrent/TTT는 이 비대칭이 **더 날카롭다**. prefill은 chunk-parallel/matmul 형태로 state를 한 번에 흡수할 수 있어 GPU에 잘 맞지만, decode는 매 토큰마다 state를 읽고→갱신하고→다시 쓰는 순차 의존이라 병렬화가 원천적으로 막힌다 [b]. KVBuffer는 바로 이 decode 국면의 **IO**를 겨냥한다(제목 그대로 “IO-aware Serving for Linear Attention”): linear-attention state 외에 request별 최근 K/V 버퍼를 두고 — paged attention에서 착안해 system-wide pool에서 block(8~16 KV) 단위로 할당 — chunkwise decoding으로 계산해 평균 메모리 접근과 decode latency를 줄이며, state 갱신을 미뤄줬다 배치로 적용(누적 KV로 write-back)한다 [arXiv:2605.19049][a]. 즉 순차 recurrence를 작은 chunk 병렬로 재구성해 대역폭 병목을 완화하는 것 — decode가 state-bandwidth-bound임을 전제로 한 설계다 [b]. Kimi Linear는 아키텍처 쪽에서 같은 병목을 공략해 KDA:full-attention = 3:1 hybrid로 장문 생성 시 KV/메모리를 최대 75% 줄이고 1M context에서 decode throughput을 최대  $6\times$  끌어올린다 [arXiv:2510.26692][a]. Nemotron-H도 Mamba-Transformer hybrid(self-attention 약 8%만 남기고 나머지를 Mamba-2/FFN로 교체)로 토큰당 constant computation·constant memory를 확보해 유사한 constant-state decode 이득을 노리며, 동급 Transformer 대비 최대  $3\times$  빠른 inference를 보고한다 [arXiv:2504.03624][a].

### 왜 hybrid의 P/D disaggregation이 더 어려운가

DistServe·Splitwise·Sarathi-Serve의 성공은 한 가정 위에서 있다 — prefill과 decode 사이에 오갈 상태가 **KV cache 하나뿐이고, 그것이 layer 단위로 잘 정의된 텐서**라는 것. DistServe는 P/D를 서로 다른 GPU에 분리해 간섭을 없애고 TTFT/TPOT를 독립 최적화해, 지연 제약을 지키면서 **최대  $7.4\times$  더 많은 request(또는  $12.6\times$  더 빠른 SLO)**를 처리한다 [arXiv:2401.09670][a]. Splitwise는 KV를 layer별로 계산되는 즉시 asynchronous 전송해 prefill 연산과 overlap시켜, 동일 비용·전력 예산에서 최대  $2.35\times$  throughput을 얻는다 [arXiv:2311.18677][a]. Sarathi-Serve는 chunked prefill로 decode의 arithmetic-intensity slack에 prefill을 끼워 stall-free 스케줄을 만든다 [arXiv:2403.02310][a].

recurrent/TTT hybrid에서는 이 그림이 흔들린다. (1) **전송 대상이 이질적**이다 — full-attention 층은 KV(길이에 비례해 커짐), linear 층은 고정 크기 state. 후자는 작아서 전송 비용은 낮지만, KV처럼 layer-by-layer로 흘러 overlap하기엔 “prefix 전체를 흡수한 뒤에야 확정되는” 단일 압축 텐서라 파이프라이닝 이득이 제한된다 [b]. (2) **TTT는 상태가 read-only가 아니다** — decode 인스턴스가 fast weight를 계속 WRITE하므로, prefill이 넘긴 state는 시작점일 뿐이고 소유권·version 관리가 disaggregation 경계를 넘어 따라가야 한다(RW-TTT의 owner/version 태깅이 정확히 이 문제) [arXiv:2605.28053][b]. (3) **prefix caching 층위가 어긋난다** — HYPIC가 보였듯 KV 재사용 primitive와 recurrent state 합성 primitive가 다르므로, disaggregated cache pool을 KV처럼 단일 규약으로 공유할 수 없다 [arXiv:2607.01299][b].

**skeptic 주석:** 위 시스템 수치들은 대개 좁은 셋업(단일 GPU, 특정 stream 수, 특정 workload)에서의 상대 배속이라 절대 goodput 이식성은 불확실하다 [c]. 특히 RW-TTT의  $9.31\times$ 는 sequential baseline 대비 값이며 per-stream replica 대비는  $3.44\times$ 로, baseline 선택에 민감하다 [arXiv:2605.28053][a]. hybrid P/D

disaggregation을 end-to-end로 검증한 대규모 프로덕션 보고는 아직 얇아, 여기 서술한 난점 (1)~(3) 상당수는 메커니즘상 추론에 기댄다 [c].

**Part II 요약.** 세 병목은 한 뿌리의 세 그림자다 — **파라미터로 안 커지고**(유한 메모리 용량), **학습이 병렬화가 안 되고**(순차 inner-loop), **서빙이 배치가 안 된다**(request-owned mutable state). 근거는 각각 용량 이론·FLOPs util·batching 불변식 붕괴다.

## Part III — 해법 3종 (무엇을·어떻게·왜 working)

### 6. 해법 ① — model-size scaling 해결

**병목 재확인: 왜 model-size가 TTT/Titans의 아킬레스건인가**

TTT·Titans 계열의 test-time memory는 근본적으로 “고정 크기 state에 컨텍스트를 online으로 압축”하는 구조다. 이 압축의 표현력은 (i) state의 크기(용량)와 (ii) 그 state를 채우고 읽는 projection 파라미터의 양에 묶여 있다 [b]. 그런데 큰 Transformer가 대규모 코퍼스로 축적한 지식은 attention의 무손실 KV-cache와 거대한 파라미터에 담겨 있어, 순진하게 recurrent 모델을 from-scratch로 키우면 그 규모를 재현하는 데 다시 방대한 토큰이 든다 [b]. 즉 “규모 병목”은 두 갈래다 — 이미 존재하는 대형 모델의 규모를 어떻게 **값싸게 상속**하느냐, 그리고 recurrent state 자체의 **용량·파라미터를 어떻게 성장**시키느냐. 아래 해법들은 이 두 갈래를 각각 공략한다.

#### 규모 상속 (1): 큰 Transformer를 distill/linearize해서 규모를 그대로 물려받기

가장 직접적인 해법은 “새로 키우지 말고 이미 큰 모델을 변환”하는 것이다. MOHAWK(Phi-Mamba)는 attention과 SSM을 모두 “token-mixing 행렬을 유도하는 sequence transformation”으로 동치화한 뒤, 3단계로 점진 정렬한다: (1) 각 층에서 teacher attention이 유도하는 mixing matrix와 student SSM의 mixing matrix를 맞추고(matrix orientation), (2) 층별 hidden state를 맞추고(hidden-state alignment), (3) 마지막으로 전체 모델을 end-to-end distill한다 [a] [arXiv:2408.10189]. 이 정렬 덕분에 Phi-Mamba는 **단 3B 토큰**만으로, 비슷한 급의 기존 open non-Transformer 모델 대비 **33× 적은 토큰 예산**으로 평균 정확도를 약 +5% 끌어올렸다 [a] [arXiv:2408.10189]. 왜 working하는가 — 병목이던 “지식 축적”을 teacher가 이미 끝냈고, student는 그 지식을 담은 **함수 형태만 바꾸면** 되므로 데이터가 극적으로 절약된다 [b].

Llamba는 MOHAWK를 스케일업해 Llama-3.x를 Mamba-2로 옮긴 1B/3B/8B 계열로, 유사 규모 모델의 통상 학습량 대비 **0.1% 미만**의 데이터로 벤치마크 성능을 유지하면서 추론 throughput과 배치 크기를 크게 키웠다(H100에서 8192-토큰 생성 기준 Llama-3.1-8B 대비 최대 12× throughput) [a] [arXiv:2502.14458]. 이는 “distill 경로가 특정 크기의 우연이 아니라 8B까지 스케일한다”는 증거다 [b].

Linear-attention 계열에서는 LoLCATs가 이 접근의 상한을 보여준다. softmax attention을 근사 linear attention으로 바꾸되, 먼저 output MSE로 linear attention이 softmax를 **모사(attention transfer)**하도록 학습하고, 남은 오차만 LoRA로 보정한다. 이 2단계 분해 덕에 파라미터의 약 0.2%만 갱신하고 **40M 토큰**으로 최초의 linearized 70B·405B LLM을 만들었다(선행연구 대비 50× 큰 규모) [a] [arXiv:2410.10254]. 5-shot MMLU에서 linearized-원본 격차를 70B·405B 각각 **77.8%·78.1%** 좁혔고, 8B·7B급에서는 **+20점 이상** 회복했다 [a] [arXiv:2410.10254]. working 메커니즘의 핵심은 “attention을 통째로 재학습하지 않고, **분포 매칭(값싼 MSE)** + **저랭크 보정(값싼 LoRA)**으로 분리”한 점이다 — 규모가 클수록 full-finetune이 불가능한데, 이 분해가 비용을 크기와 거의 무관하게 만든다 [b].

Liger는 여기서 한 걸음 더 나아가 **추가 feature-map 모듈 없이** 기존 가중치만 재활용한다. 새 gating 모듈을 별도 학습하는 대신, 사전학습된 **key projection 가중치**를 재활용해 gated recurrent 구조의 gate를 구성하고, LoRA 경량 미세조정만 얻는다. 이 방식으로 **0.02% 사전학습 토큰**에서 원본 성능의 **93%**를 회복했으며(intra-layer hybrid인 **Liger Attention**을 함께 쓴 수치), 이는 layer 내부에 소량의 attention을 섞어 성능을 더 끌어올린 결과다 [a] [arXiv:2503.01496]. 왜 그럴까 — key 가중치는 이미 “어떤 토큰이 중요한가”의 통계를 담고 있어, gate(무엇을 기억/망각할지)의 좋은 초기값이 되기 때문이다 [b].

## 규모 상속 (2)는 곧 한계: state 용량을 키워야 한다

distill/linearize는 “규모를 물려받되 고정 state로 압축”하므로, retrieval·reasoning처럼 정보를 많이 보존해야 하는 과제에서 열화한다(SSE도 이 한계를 명시적으로 지적한다) [a] [arXiv:2507.16577]. 따라서 두 번째 갈래 — **용량·파라미터 성장** — 이 필요하다.

Atlas는 memory의 표현 용량을 **polynomial feature mapping**으로 확장한다. 선형 memory는 key의 1차 외적만 저장하지만, 다항 feature는 고차 상호작용을 담아 같은 state 크기로도 저장 가능한 연관(key→value 쌍)의 수를 늘린다 [b]. 여기에 마지막 토큰만이 아니라 최근 c개 토큰(컨텍스트 창)을 함께 보고 memory를 갱신하는 **Omega rule**, 2차 정보를 근사하되 병렬화되는 **Muon optimizer**, 그리고 깊은(deep) neural memory를 결합해, 10M 컨텍스트 BABILong에서 **Titans 대비 +80% 정확도**를 보고한다 [a] [arXiv:2505.23735]. 용량이 병목이던 지점을 “state 크기를 키우지 않고 feature 차원과 갱신 규칙을 바꿔” 완화한 것이다 [b].

**Sparse State Expansion(SSE)**는 파라미터와 용량을 분리(decouple)한다. state 갱신을 “정보 분류” 문제로 재정식화해 softmax top-k hard classification으로 **row-sparse update**를 수행하고, state를 여러 partition으로 확장한다. 각 partition이 서로 다른 정보 부류를 담아 간섭(inter-class interference)이 줄고, partition 수를 늘리면 **파라미터를 늘리지 않고도 유효 용량(state capacity)이 커진다** [a] [arXiv:2507.16577]. in-context retrieval·수학 추론 벤치마크에서 강한 성능을 보였고, RL 학습 후 2B SSE-H 모델은 소형 추론모델 중 AIME24 64.5 / AIME25 50.2로 SOTA급을 기록했다 [a] [arXiv:2507.16577].

## 파라미터 성장: 모델이 스스로 더 커지고 더 깊이 학습하기

세 번째 갈래는 아예 파라미터·깊이를 성장시킨다. **Sleep**(Language Models Need Sleep)은 파라미터를 실제로 **확장(parameter expansion)**하면서, 작은-self가 in-context로 축적한 취약한 단기 memory를 더 큰 network로 **상향 distill**하는 **Knowledge Seeding**으로 용량을 키운다. 여기에 **Dreaming** 단계에서 RL로 합성 데이터 커리큘럼을 생성해 새 지식을 rehearse하며 자기 자신을 재귀 개선한다 [a] [arXiv:2606.03979]. 취약한 in-context 지식을 안정적 파라미터 지식으로 consolidation한다는 점에서, TTT의 “고정 state 압축” 한계를 파라미터 성장으로 우회하는 접근이다 [b].

**HOPE**(Nested Learning)는 모델을 **다층 중첩 최적화 문제**로 본다. optimizer조차 gradient를 압축하는 associative memory로 재해석하고, **self-modifying** 모듈(hidden state가 곧 자기 가중치이며 optimizer로 갱신됨)과 **continuum memory system(CMS)**(서로 다른 주기로 갱신되는 memory बैं크들의 연속 스펙트럼)을 결합해, “nested depth”라는 성장 축을 연다 — 파라미터 폭이 아니라 **학습 규칙의 깊이**로 용량을 키우는 셈이다 [a] [arXiv:2512.24695]. 언어모델링·장기컨텍스트 관리에서 표준 Transformer·최신 recurrent 대비 우수한 성능을 보고한다 [a][b] [arXiv:2512.24695]. (HOPE·Atlas·Sleep·Titans는 모두 Behrouz 등 동일 연구계보에서 나온 Titans 변주다 [a].)

## hybrid: attention을 일부 남겨 규모 병목을 우회

마지막으로, 여러 distill 계열이 공통으로 채택하는 실용 전략은 **일부에 softmax attention을 유지**하는 것이다. 다만 결이 두 가지로 갈린다 — (i) **inter-layer** hybrid는 소수의 층을 통째로 attention으로 남기고(예: Hybrid Phi-Mamba의 소수 attention 층), (ii) **intra-layer** hybrid는 한 층 안에서 linear recurrence와 attention을 섞는다(예: Liger Attention) [a] [arXiv:2408.10189][arXiv:2503.01496]. 어느 쪽이든 recurrent state가 못 담는 정밀 retrieval을 소수의 attention 경로가 담당해, 전체를 linear로 바꿀 때의 열화를 값싸게 메운다 [b].

## skeptic 각주

이 계열의 baseline인 Titans의 재현성은 논쟁적이다 — Titans Revisited는 경량 재구현에서, chunking을 쓴 Titans가 확립된 baseline을 **항상 이기지는 못한다**고 비판적으로 보고한다(원 논문은 코드 미공개·서술 모호성으로 재현이 어려웠다) [a] [arXiv:2510.09551]. 다만 같은 연구는 Titans의 **Neural Memory 구성요소 자체는 attention-only 대비 일관되게 성능을 개선**한다고도 확인한다 [a] [arXiv:2510.09551]. 따라서 “Atlas가 Titans 대비 +80%” 같은 상대 수치는 Titans baseline의 강도(및 구현 선택)에 의존하며 절대적 우열로 읽어선 안 된다 [c]. 또한 distill/linearize의 “N% 토큰으로 회복” 수치는 대개 특정 벤치마크(MMLU 등) 기준이라 long-context retrieval까지 일반화된다는 보장은 없다 [c].

## 7. 해법 ② — training parallel/batch scaling 해설

TTT·Titans 계열의 학습 병목은 결국 **fast weight**를 매 토큰(또는 매우 작은 minibatch)마다 순차적으로 갱신하는 **inner-loop recurrence**에서 나온다. 갱신량 자체는 작은 matrix-vector 연산이라 GPU의 tensor core를 채우지 못하고 memory-bound에 갇히며, 게다가 순차 의존성 때문에 sequence 길이 방향으로 병렬화가 되지 않는다. 이 절의 해법들은 (i) 갱신 단위를 키워 연산을 compute-bound로 옮기거나, (ii) 순차 recurrence를 sequence 길이에 대해 병렬 가능한 형태로 재정식화하거나, (iii) 그 재정식화를 실제 하드웨어 커널로 구현하는 세 갈래로 나뉜다. 갈래마다 “왜 정확도를 지키는가”의 근거가 다르다는 점이 핵심이다.

**Large chunk로 FLOPs util 끌어올리기 (LaCT).** 기존 TTT layer가 16·64 토큰마다 fast weight를 갱신하는 이유는 online 학습의 신선함(recency)을 살리기 위함인데, 이 작은 minibatch가 바로 저효율의 원인이다. 각 갱신이 너무 작아 GPU peak FLOPs의 5% 미만만 쓰는 경우가 흔하다 [a][arXiv:2505.23884]. LaCT는 chunk를 2K~1M 토큰으로 극단적으로 키운다. chunk 내부는 하나의 큰 batch로 묶여 병렬 처리되므로 연산이 compute-bound로 전환되고, hardware utilization이 <5%에서 A100 기준 최대 ~70%까지 올라간다 — 그것도 수십 줄짜리 순수 PyTorch로 [a][arXiv:2505.23884]. 여기서 얻는 두 번째 이득이 중요한데, chunk가 커져 갱신 1회의 연산 예산이 커지자 비로소 SwiGLU-MLP 같은 **비선형·고표현력 fast-weight**를 감당할 수 있게 되어, nonlinear state 크기를 모델 파라미터의 최대 ~40%까지 키워 state capacity가 커진다 [a][arXiv:2505.23884]. 대가는 명확하다. 한 chunk 안의 토큰들은 서로의 갱신을 보지 못하므로 recency 해상도가 거칠어진다. 이는 근사(토큰 단위 순차 recurrence와 수학적으로 동일하지 않음)이며, LaCT는 늘어난 state capacity로 이 손실을 상쇄한다고 보는 것이 합리적이다 [b].

**WY/Householder chunkwise-parallel (DeltaNet).** DeltaNet의 delta rule은 매 스텝 state를  $S \leftarrow S(I - \beta t_{kt} t_{kt}^T) + \dots$  형태로 갱신한다. 이 데이터 의존적 전이행렬  $(I - \beta t_{kt} t_{kt}^T)$ 는 사실상 (generalized) Householder 변환에 해당하고, 스텝마다 값이 달라지기 때문에 GLA·Mamba 같은 단순 대각/스칼라 전이와 달리 associative scan으로 곧장 병렬화되지 않는다 — 이것이 DeltaNet이 표현력은 높지만 학습이 느렸던 근본 이유다 [a][arXiv:2406.06484]. 해법은 chunk 내부의 Householder 곱을 **WY representation**으로 재매개화하는 것이다. WY 형태는 여러 Householder 행렬의 곱을 소수의 행렬로 압축해 표현하므로, 토큰마다 matrix-sized hidden state를 실체화(materialize)할 필요 없이 chunk 단위 행렬 곱으로 한꺼번에 계산할 수 있다 [a][arXiv:2406.06484]. 결정적으로 이 재정식화는 **정확히 동치**다 — 근사가 아니라 같은 계산을 다른 순서로 묶은 것이므로 정확도 손실이 없고, chunk 경계 간 state 전달도 정확하다 [b]. 그 결과 DeltaNet을 1.3B 규모(100B 토큰 학습)로 확장했고 Mamba·GLA를 상회했다 [a][arXiv:2406.06484].

**계층적 chunk로 속도-정확도 분리 (TNT).** chunkwise 병렬화에는 근본 딜레마가 있다. chunk가 크면 빠르지만 memory 갱신이 성기어 정확도가 떨어지고, 작으면 반대가 된다. 기존 방법은 이 chunksize를 하나로 고정해 어중간한 타협을 강요당한다 [a][arXiv:2511.07343]. TNT는 이 둘을 **두 단계로 분리**한다. 1단계 pre-training은 계층적 memory를 쓰는데, global 모듈이 크고 하드웨어 친화적인 chunk로 장거리 문맥을 처리하고 여러 local 모듈이 병렬로 세부를 담당한다. 2단계는 짧은 fine-tuning으로 local memory만 작은 고해상도 chunksize로 적응시켜 정확도를 회복한다 [a][arXiv:2511.07343]. 학습 대부분을 큰 chunk로 태워 속도를 얻고, 정확도를 결정하는 부분만 국소적으로 미세 갱신하는 구조다. “가장 정확한 baseline 설정” 대비 최대 17배 빠르면서 정확도도 동시에 더 높다고 보고한다 [a][arXiv:2511.07343]. (skeptic: 17배는 “가장 정확한=가장 느린 고정밀 설정” 대비 수치라 baseline 선택에 민감하며, 저자 스스로 “up to”로 표기한다 [b].)

**비선형 recurrence의 병렬화 (DEER·ParaRNN·Predictability).** 위 해법들이 특정 구조(선형/Householder)를 이용한다면, 이 갈래는 임의의 비선형 recurrence  $st = ft(st-1)$ 를 통째로 병렬화한다. DEER는 전체 sequence의 recurrence를 하나의 fixed-point 문제로 세우고 Newton 반복의 병렬 형태로 푼다 — quadratic 수렴이라 소수 iteration이면 되고, 출력은 수치정밀도 내에서 순차 계산과 동일하다 [a][arXiv:2309.12252]. 아키텍처에 특별한 구조를 요구하지 않아 적용 범위가 넓고, GPU 평가 최대 3자리수, 학습 10배 이상 가속을 보고한다 [a][arXiv:2309.12252]. 다만 Newton 스텝이 state size에 cubic이고 수치 불안정이 있다는 한계가 있으며, 이 문제를 정면으로 다뤄 안정·확장성을 개선한 것이 후속의 quasi-DEER/ELK 계열이다 [a][arXiv:2407.19115]. ParaRNN은 이 아이디어를 LLM 규모로 밀어붙여, sequence 전체를 하나의 방정식계로 보고 Newton 반복에 custom parallel reduction을 결합, naïve 순차 대비 최대 665배 가속으로 7B LSTM/GRU를 Transformer·Mamba2에 필적하는 perplexity로 학습한다 [a][arXiv:2510.21450]. Predictability 논문은 “왜 어떤 비선형 모델은 잘 병렬화 되고 어떤 것은 안 되는가”를 이론적으로 답한다 — 시스템의 largest Lyapunov exponent(LLE)로 측정한 예측 가능성이 최적화 문제의 조건수(PL 상수)를 결정하고, 예측 가능한 시스템은 항상 well-conditioned가 되어 상태 궤적을 최악의 경우에도  $O((\log T)^2)$ 에 계산할 수 있는 반면 카오스적 시스템은 조건수가 붕괴해 병렬 수렴이 쓸모없이 느려진다 [a][arXiv:2508.16817]. 즉 예측 가능한(카오스적이지 않은) recurrence일수록 fixed-point 반복이 적게 든다는 메커니즘적 근거다 [a]. 이 세 연구의 정확도 보장은 “같은 방정식의 fixed point”라는 데서 나오지, 근

사에서 나오지 않는다 [b]. (단, 이 갈래는 일반 비선형 RNN(LSTM/GRU 등)에서 실증되었을 뿐 TTT/Titans의 inner-loop 자체에 직접 적용된 결과는 아직 미명시이며, TTT류에 대한 관련성은 “동일한 비선형 순차 갱신을 병렬화하는 일반 도구”라는 수준이다 [c].)

**하드웨어 커널 (Comba-Tiled FLA).** 마지막 갈래는 위 수학을 실제 GPU 처리량으로 바꾼다. TFLA는 FLA의 chunkwise 병렬 형태가 chunk 크기 제약 때문에 chunk마다 중간 state를 GPU 메모리에 materialize해 arithmetic intensity가 낮고 IO가 큰 문제를 지적한다 [a][arXiv:2503.14376]. 해법은 각 chunk 내부에 **또 한 겹의 sequence 병렬화**를 도입해 arbitrary large chunk를 허용하면서 arithmetic intensity를 올리고 메모리·IO와 연산을 균형 잡는 것이다 [a][arXiv:2503.14376]. 이렇게 만든 mLSTM(xLSTM matrix-memory) 커널은 고도로 최적화된 Flash Attention·Linear Attention·Mamba 커널을 앞선다 [a][arXiv:2503.14376]. Comba는 bilinear RNN(state와 key의 상호작용, closed-loop control 관점 — Gated DeltaNet·TTT·RWKV-7 계열이 구조적으로 여기 속한다)을 위한 chunkwise Triton 커널을 구현해 forward에서 Gated-DeltaNet 대비 40% 빠르다 [a][arXiv:2506.02475]. 이 커널들은 알고리즘의 수학을 바꾸지 않으므로 정확도에 대체로 중립적이고, 앞선 재정식화가 실전 학습 속도로 실현되는 마지막 고리 역할을 한다 [b].

**Sources:** - [arXiv:2505.23884](#) LaCT / Test-Time Training Done Right - [arXiv:2406.06484](#) Parallelizing DeltaNet (WY/Householder) - [arXiv:2511.07343](#) TNT - [arXiv:2309.12252](#) DEER - [arXiv:2407.19115](#) Towards Scalable and Stable Parallelization of Nonlinear RNNs (quasi-DEER/ELK) - [arXiv:2510.21450](#) ParaRNN - [arXiv:2508.16817](#) Predictability Enables Parallelization of Nonlinear State Space Models - [arXiv:2506.02475](#) Comba - [arXiv:2503.14376](#) Tiled Flash Linear Attention

## 8. 해법 ③ — serving/P/D batch inference 해결

문제의 재정의 — **serving이 암묵적으로 가정하던 두 불변식이 깨진다**

기존 LLM serving 처리량의 토대는 두 가지 암묵적 불변식이다. 첫째, **가중치는 공유·정적(static)**이라 임의의 요청을 한 배치로 묶어도 서로 간섭이 없다. 둘째, **KV는 per-token 단위로 재사용 가능**하므로 prefix가 겹치면 앞부분 계산을 건너뛴다. TTT/Titans 계열은 이 둘을 동시에 깨다. TTT는 생성 중에 fast weight·low-rank delta-streaming learner state 같은 **request-owned state**를 읽고(READ) 쓰기(WRITE) 때문에, 이는 “가중치 공유” 가정을 위반한다[a][arXiv:2605.28053]. 또한 recurrent/SSM 층은 상태를 **in-place로 덮어쓰기** 때문에 부분 겹침에 대한 roll-back이 불가능하고, 그 결과 exact-match hit만 허용되어 per-token KV 재사용 프리미티브가 그대로 이식되지 않는다[a][arXiv:2411.19379]. 따라서 이 갈래의 serving 해법은 전부 “정합성을 지키면서(= 한 요청의 WRITE가 다른 요청의 상태를 오염시키지 않고, roll-back된/stale 상태를 서빙하지 않으면서) 어떻게 배치·재사용을 복원할 것인가”라는 한 문제의 변주다.

### RW-TTT — request-owned state를 오염 없이 배치화

naive 배치는 왜 틀리는가. TTT 스트림마다 fast weight가 다른데 이를 한 커널에 묶으면 한 스트림의 WRITE가 다른 스트림의 상태에 새어 들어가 상태를 오염시킨다. 순차 실행은 정합하지만 GPU를 놀린다. RW-TTT는 이를 **read-write serving** 문제로 정식화해, 각 decode step에 owner·version·READ/WRITE effect를 태깅하고 **호환되는 phase만** 배치한 뒤 갱신은 owner에게만 commit한다[a][arXiv:2605.28053]. 메커니즘의 핵심은 공유 가중치로 forward하는 READ 단계는 자유롭게 배치하되, WRITE 단계는 owner·version으로 격리해 커밋 순서를 보존하는 것이라, 배치가 상태 semantics를 바꾸지 못한다[b]. 단일 GPU·8개 In-Place-TTT fast-weight 스트림에서 274.61 aggregate tok/s로, 순차 서빙 대비 9.31×, 동일 메모리 예산의 per-stream replica 대비 3.44× 빠르며, 장문맥 벤치 RULER에서 동작을 보존한다[a][arXiv:2605.28053]. version 태그가 곧 정합성 증거라는 점이 이 설계가 working하는 이유다[b].

### hybrid/recurrent를 위한 prefix caching — 재사용 프리미티브 자체를 바꾼다

Marconi가 진단한 병목: hybrid 모델의 in-place 상태 갱신은 cache 엔트리의 부분 roll-back을 원천적으로 막아 **exact-match hit만** 허용하고, 그 결과 시퀀스마다 큰 엔트리가 폭증하되 재사용률은 낮다[a][arXiv:2411.19379]. Marconi는 recency뿐 아니라 **재사용 예측(forecast)**까지 반영한 admission/eviction 정책으로 캐시 후보를 선별해, SOTA prefix caching 대비 최대 34.4× 높은 token hit rate와 71.1%(또는 617 ms) 낮은 TTFT를 얻는다(MLSys’25)[a][arXiv:2411.19379].

**Sparse Prefix Caching**는 문제를 더 근본적으로 재구성한다. recurrent 층은 전체 토큰 히스토리 없이 **단 하나의** 저장된 state에서 **resume**할 수 있다는 성질을 이용해, 최소한 checkpoint 위치에만 정확한 recurrent state를 저



장하고 hit 시 가장 깊은 checkpoint에서 이어받아 나머지 suffix만 **정확히 recompute**한다. checkpoint 배치를 overlap-depth 분포 하의 배치 문제로 형식화해 O(NM) DP로 최적해를 구하며, QuALITY-System Prompts 등에서 고정 예산 baseline을 checkpoint를 더 적게 쓰면서 지배(dominate)한다[a][arXiv:2605.05219]. suffix를 exact recompute하므로 근사가 아니라 **정확성이 정의상 보존된다**[a].

HYPIC는 RAG/agent적 처럼 비연속 segment를 조립하는 워크로드에 position-independent caching(PIC)을 hybrid로 확장한다. 문제는 per-token KV 재사용이 per-request recurrent state로 옮겨가지 않는다는 것. HYPIC는 linear-attention 층에서 **segment-cumulative transition operator**라는 빠져 있던 대수적 프리미티브를 찾아, 각 segment의 zero-start end-state와 함께 캐시함으로써 독립 캐시된 segment들을 **near-exact·상수시간으로 state 합성**한다. full-attention 층은 경계마다 작은 seam window만 recompute해 cross-segment lookback을 복원한다. 결과적으로 4개 hybrid-attention 모델·5개 워크로드에서 TTFT를 평균  $3.25\times$  단축, 동일 1초 TTFT SLO에서 QPS를  $1.66\times$  향상, full-recompute 대비 정확도 격차 평균 1.71 point, cold 요청에서는 8-instance 시 TTFT  $5.7\times$  가속을 달성한다[a][arXiv:2607.01299]. transition operator의 합성 가능성이 정합성의 대수적 근거다[b].

### IO-aware 서빙과 최소 state — 병목이 compute가 아니라 memory access일 때

linear attention의 decode는 산술량이 작아 오히려 **메모리 접근이 병목**이 되기 쉽다. KVBuffer는 최근 K/V를 buffer해 chunkwise decoding·speculative decoding 검증·short-context decoding 등 상황별로 계산 형태를 유연하게 바꿔 불필요한 memory access를 줄이고 hybrid 서빙 효율을 높인다[a][arXiv:2605.19049]. 여기서 정합성은 buffer가 recurrent 재귀식과 수학적으로 동치인 형태만 취하기 때문에 보존된다[b].

state를 크게 하지 않는 것 자체도 serving 해법이다. **In-Place TTT**는 새 모듈 없이 모든 MLP 블록의 최종 projection matrix를 fast weight로 재활용하고 NTP-정렬 목적함수 + chunk-wise 갱신을 쓴다(ICLR 2026 Oral)[a][arXiv:2604.06169]. 요청당 추가로 관리·격리해야 할 state 표면을 최소화한다는 점에서, 실제 RW-TTT의 벤치마크 스트림이 바로 이 In-Place-TTT라는 사실[a][arXiv:2605.28053]은 “최소 state가 배치를 현실화한다”는 관계를 시사한다[b]. 다만 fast weight를 어디에 두느냐가 정확도·격리 비용의 trade-off라는 점은 아직 열린 설계공간이다[c].

### hybrid stack으로 KV·state를 구조적으로 줄이기 + recurrent에 맞춘 P/D 분리

가장 직접적인 처리량 이득은 아키텍처에서 온다. Nemotron-H는 self-attention 대다수를 Mamba로 대체해 토큰당 상수 compute·상수 memory를 확보, 동급 정확도에서 최대  $3\times$  빠른 추론을 낸다[a][arXiv:2504.03624]. Nemotron Nano 2(9B)는 같은 hybrid로 reasoning 설정(8k input/16k output)에서 최대  $6\times$  처리량을 내고, 단일 A10G(22GiB, bf16)에서 128k를 처리한다[a][arXiv:2508.14444](이보다 세분화된 설정별 배수는 초록에 명시되지 않음[c]). Kimi Linear는 KDA(Gated DeltaNet에 fine-grained gating 추가) + MLA를 uniform 3:1로 섞어 KV cache를 최대 75% 줄이고 1M 컨텍스트에서 decoding 처리량 최대  $6\times$ 를 얻는다[a][arXiv:2510.26692]. KV가 줄면 배치 크기 상한이 커져 처리량이 오르는 것이 메커니즘이며, 정합성은 아키텍처가 그렇게 학습·서빙되므로 정의상 보존된다[b].

마지막으로 **P/D(prefill/decode) 분리**를 recurrent에 맞춰 재설계하는 갈래가 있다. 표준 P/D 분리는 KV cache를 네트워크 payload로 전송하는데, hybrid/SSM에서는 전송 대상이 per-token KV가 아니라 **compact한 recurrent state**로 바뀐다 — 이는 전송량을 줄이는 기회이자, 상태 버전을 정확히 넘겨야 하는 정합성 과제다[b]. vLLM은 hybrid SSM-FA 모델의 disaggregated serving을 NIXL 기반 KV connector 확장으로 실제 구현했고(FA층과 SSM층의 서로 다른 state layout·size를 함께 처리, bf16에서 요청당 약 50MB 불필요 전송 제거)**a**, SpectrumKV처럼 P/D 전송을 per-token 혼합정밀도(FP16/INT8/INT4)로 압축하는 연구도 이 축과 접점을 갖는다(단, 이는 attention KV 대상이며 recurrent state로의 직접 확장은 미명시)[c][arXiv:2606.08635]. 요컨대 recurrent-aware P/D 분리는 “state를 언제·어느 버전으로 넘길지”를 명시적으로 다뤄야 정합하며, 이 부분은 아직 표준화 이전의 활발한 갈래다[b].

**Part III 요약.** 해법도 세 방향이다 — (규모) 큰 Transformer의 규모를 **distill로 물려받기**, (학습) 순차성을 큰 **chunk·병렬화로 접기**, (서빙) **배치·캐시 프리미티브 자체를 바꾸기**. 공통적으로 ‘표현력을 지키며 순차성/용량 비용을 줄인다’가 working의 조건이다.

## Part IV — memory device 집중 조명

### 9. memory device 집중 조명 ① — 한계

TTT/Titans 계열은 sequence를 fixed-size의 fast-weight(혹은 recurrent state)  $S$ 에 눌러 담고, 그  $S$ 를 test time에 gradient step으로 갱신한다. 이때  $S$ 는 두 층위에서 동시에 “memory device”다. 하나는 **정보이론적 저장 장치** — key→value 연상을 담는 associative memory로서의  $S$ 이고, 다른 하나는 **물리적 저장 장치** — HBM 위에 실제 bytes로 상주하며 매 decode step마다 읽고 써야 하는 tensor로서의  $S$ 다. 이 섹션은 두 층위 각각의 한계가 어떻게 발생하고, 그것이 앞서 정리한 세 병목의 진원지가 되는지를 메커니즘 단위로 짚는다.

#### (i) 신경 메모리의 유한 용량: interference와 retrieval error의 성장

가장 단순한 신경 메모리는 outer-product(Hebbian) rule로 갱신되는 선형 행렬  $M \in \mathbb{R}^{d \times d}$ , 즉  $M += vk^\top$ 이다. 파라미터는  $d^2$ 개이지만, **완벽히 복원 가능한 key의 수는  $\sim d$ 개에 불과하다** [b]. 이유는 retrieval이  $\hat{v} = Mk = \sum_i v_i(k_i^\top k)$ 로 계산되기 때문이다. key들이 서로 직교하면  $k_i^\top k_j = \delta_{ij}$ 라 간섭 없이 target만 뽑히지만, 저장 항목이  $d$ 를 넘으면  $d$ 차원 공간에 직교 벡터를 더 넣을 수 없어  $k_i^\top k_j \neq 0$ 의 **cross-talk(interference)**이 필연적으로 발생하고, retrieval error가 저장 개수에 따라 성장한다 [b]. (TTT/DeltaNet 계열이 실제로 쓰는 delta rule  $M += (v - Mk)k^\top$ 은 여기에 error-correction 항을 더해 간섭을 완화하지만, 아래의 유한 용량 한계 자체를 없애지는 못한다 [b].) 이것이 “고정 용량의 긴 문맥 열화”의 근본 메커니즘이다: 문맥이 길어질수록 같은  $S$ 에 더 많은 연상을 눌러 담게 되고, 용량 초과 지점부터 오래된/충돌하는 key의 값이 뭉개진다.

용량을 키우는 두 갈래가 있고, 둘 다 근본 한계를 없애지 못한다. 첫째, **feature map을 키우는 길**. Modern continuous Hopfield network는 log-sum-exp 에너지에서 유도된 softmax 형태의 업데이트로 pattern을 차원에 지수적으로 저장하고 exponentially small retrieval error를 갖는다고 주장하며, 이 업데이트가 곧 transformer attention과 등가임을 보인다 [a][arXiv:2008.02217]. 즉 “attention이 강한 recall을 갖는 이유”는 그것이 사실상 지수 용량 associative memory이기 때문이다 [b]. 하지만 이 지수 용량은 **모든 pattern을 명시적으로 KV로 들고 있을 때**의 이야기이고, 이를 fixed-size state로 압축하는 순간 사라진다 [b]. Atlas는 memory capacity가 “메모리의 architecture와 입력의 feature mapping에 의해 bound된다”고 명시하고, polynomial(고차) feature map으로 유효 key 차원을 키워 용량을 올린다 [a][arXiv:2505.23735]. Titans는 선형 행렬 대신 **deep neural memory(MLP)**를 써서 용량을 비선형으로 확장한다 [a][arXiv:2501.00663]. 둘 다 “ $\sim d$ ” 상한을 밀어낼 뿐, 유한성 자체를 깨지는 못한다 [b].

둘째, **용량을 능동적으로 재배분하는 길**. Conformal-sympow는 sympow transformer의 recurrent state가 유한하기에 학습/평가 context를 늘리면 정보 보존이 무너져 성능이 열화된다는 점을 출발점으로, data-dependent multiplicative gating으로 용량을 “비워주고” data-dependent rotary embedding으로 적응적으로 저장한다 [a][arXiv:2503.03269]. 이는 흥미로운 확증이다: 용량이 유한하다는 사실은 부정할 수 없으니, **무엇을 잊을지를 학습하는 것이 유일한 대응**이라는 것. Titans의 surprise-기반 저장과 forget gate도 정확히 같은 논리다 — 유한 저장 장치를 전제로 한 admission/eviction 정책 [b][arXiv:2501.00663].

이 유한성의 이론적 하한이 **Impossibility Triangle**이다. Efficiency(step당 계산이 sequence 길이와 무관), Compactness(state 크기가 길이와 무관), Recall(길이에 비례하는 사실 수를 복원)의 세 가지는 셋 중 둘만 동시에 가능하다는 결과로, Online Sequence Processor 추상화 위에서 형식화된다: Efficiency와 Compactness를 동시에 만족하는 시스템은 정확도  $1 - \epsilon$ 로 많아야  $O(\text{poly}(d)/\log V)$ 개의 key-value만 복원할 수 있어  $T \rightarrow \infty$ 에서  $o(T)$ 가 되고, 이는 길이-비례 Recall과 모순된다 [a][arXiv:2605.05066]. TTT/Titans는 Efficiency+Compactness를 택한 코너이므로 **Recall이 길이에 비례해 자랄 수 없다** — 이는 튜닝으로 없앨 수 있는 결함이 아니라 memory device를 fixed-size로 고른 대가다 [b]. Mamba-Transformer hybrid 분석도 경험적으로 이를 뒷받침한다: 순수 SSM은 fixed state로 fine-grained recall에 취약하고, attention layer를 섞어야 recall이 회복되며, 두 층을 블록 안에 순차로 쌓는 sequential hybrid는 짧은 문맥에, 병렬로 두고 출력을 합치는 parallel hybrid는 긴 문맥에 유리하다 [a][arXiv:2510.26912]. 즉 recall을 사려면 다시 길이-비례 attention( $\approx KV$ )을 사 오는 셈이다 [b].

#### (ii) physical memory 한계: HBM 점유·decode 대역폭·batch×state

같은  $S$ 를 이번엔 bytes로 보면 세 병목의 물리적 원인이 드러난다.

**병목 1 — state footprint의 HBM 점유.** Compact state를 자랑하는 linear/TTT 계열조차, deep neural memory나 고차 feature map으로 용량을 올린 순간 state 자체가 커진다. KVBuffer는 linear attention의 state가 “per-token key/value보다 훨씬 크다”고 명시하며, 이 큰 state가 recurrent decoding의 비효율과 HBM 상주 압력의 원천임을 지적한다 [a][arXiv:2605.19049]. 반대편 attention은 KV-cache가 길이에 비례해 부풀어 HBM을 잠식한다 [a][arXiv:2605.05066]. 어느 쪽이든 memory device의 물리적 크기가 서빙 가능한 문맥·모델을 제한한다 [b].

**병목 2 — decode의 메모리 대역폭 bound.** Decode는 한 번에 한 token만 만들므로 arithmetic intensity가 낮다: 매 step마다 state(또는 전체 KV)를 HBM에서 스트리밍해 오고 소량의 곱셈만 한다. 따라서 latency는 FLOPs가 아니라 **state를 읽어 오는 대역폭**이 결정한다 [b]. KVBuffer는 바로 이 “매 decode step마다 큰 linear state를 갱신·재계산”하는 IO를 겨냥해 최근 key/value를 buffering하고 chunkwise/parallel form으로 state 갱신을 미뤄, linear attention decoding latency를 최대 45.17% 줄이고, speculative decoding에서 draft token 4개를 검증할 때 최대 동시 서빙 요청 수를 5배로 늘린다 [a][arXiv:2605.19049]. 이는 병목의 원인이 계산이 아니라 memory device의 read/write IO임을 역으로 증명한다 [b].

**병목 3 — batch × state 메모리.** 서빙 처리량은 batch를 키워 벌지만, 필요한 메모리는 (**요청당 state 크기**) × (**batch 크기**)로 곱해져 커진다. 요청당 state가 클수록 답을 수 있는 batch가 줄어 throughput 상한이 낮아진다 [b]. TTT/Titans가 문맥 열화를 막으려 용량(=state 크기)을 키우면 이 곱이 그대로 커지므로, “긴 문맥 recall”과 “높은 동시 처리량”이 물리 메모리에서 직접 충돌한다 [b]. Impossibility Triangle의 Compactness 축이 시스템 층위에서 다시 나타나는 지점이다 [b][arXiv:2605.05066].

### 종합: 하나의 장치, 두 상한

정리하면 memory device는 **정보이론적 상한(유한 용량·interference·retrieval error 성장)**과 **물리적 상한(HBM 점유·대역폭·batch × state)**을 한 몸에 지닌다. 용량을 키우려는 처방(deep memory, 고차 feature map)은 곧바로 물리 footprint를 키워 대역폭·batch 병목을 악화시키고, footprint를 줄이려는 처방(더 compact한 state)은 Impossibility Triangle에 의해 recall을 깎는다. 두 상한은 독립이 아니라 **같은 장치의 앞뒤 면**이며, 이것이 이후 섹션에서 “무엇을 저장·망각하고, 어디에 상주시킬지”를 설계 변수로 끌어올려야 하는 이유다 [b]. (다만 Titans의 재현성 논란처럼 — 공개 코드 부재와 서술 모호성 탓에 독립 재구현이 Titans가 항상 baseline을 앞서는 양상을 보고했다 [a][arXiv:2510.09551] — 개별 용량 주장은 별도 검증이 필요하며, 여기서는 상한의 존재만 근거로 삼는다 [c].)

## 10. memory device 집중 조명 ② — 개선으로 해결되는 해법

앞 절에서 fixed-size state의 근본 병목은 (1) 유한 용량에 무한 문맥을 옥여넣는 데서 오는 associative collision(간섭), (2) 그 state를 매 토큰 갱신할 때의 물리적 read/write 비용이었다. 이 절의 해법들은 device 자체를 바꾸지 않고 “memory를 어떻게 늘리고·비우고·저장하고·움직이는가”만 개선해서 정확도를 회복한다는 공통점이 있다. 갈래별로 메커니즘을 따라간다.

### (i-a) state를 키우되 FLOPs는 묶어두는 방식 — expandable/sparse state

병목의 정공법은 state를 키우는 것이지만, dense state를 키우면 매 스텝 outer-product 갱신 비용이 용량에 비례해 커진다. Sparse State Expansion(SSE)은 state 갱신을 “정보 분류(information classification)”로 재해석해, softmax 기반 top-k hard classification으로 **한 토큰이 실제로 건드리는 행(row)만 갱신하는 row-sparse update**를 도입한다. 이로써 receptive field는 넓히면서 inter-class 간섭은 줄이고, contextual state를 여러 partition으로 확장해 파라미터 수와 state 용량을 분리한다 [a][arXiv:2507.16577]. Sparse Delta Memory(SDM)는 같은 직관을 Gated DeltaNet 위에서 구현한다 — dense KV outer product를 **큰 explicit memory에 대한 sparse read/write(sparse addressing)**로 치환한다. 핵심은 isoFLOP·동일 파라미터 제약 하에서 state 용량을 orders-of-magnitude 키워도, 토큰당 소수 슬롯만 접근하므로 FLOPs가 용량에 선형으로 끌려가지 않는다는 점이며, in-context learning과 long-context retrieval에서 이득을 보인다 [a][arXiv:2607.07386]. 두 해법 모두 “용량↑ & 비용↑”의 결합을 sparsity로 끊는 것이 accuracy를 지키는 이유다 [b].

### (i-b) 압축이 아닌 정확 저장을 병치 — exact memory

압축 state는 본질적으로 lossy라, 경합하는 key-value가 많으면 앞선 사실이 덮여 needle recall이 무너진다. HOLA(A Hippocampus for Linear Attention)는 delta-rule state를 **compressive memory**로 유지하되, 옆



에 bounded **exact KV cache**를 붙인 semiparametric 구조다. 무엇을 cache할지는 delta-rule 갱신이 이미 내놓는 write 신호(write strength  $\beta \times \text{prediction residual } \|e\|$ 가 큰 토큰)로 고른다 — 즉 모델이 예측에 실패해 state를 크게 흔든 토큰만 정확 저장하고, 별도의 학습된 eviction 모듈 없이 write한다. 340M 파라미터·15B SlimPajama 토큰 학습에서 Wikitext ppl 27.32→22.92로 낮추어 full-attention Transformer++(26.88)보다도 낮고, 학습 길이의 16배인 32k RULER NIAH까지 GDN·recency cache보다 견고하다 [a][arXiv:2607.02303]. EFLA(Error-Free/Exact Flow Linear Attention)는 “exact”의 결이 다르다 — delta rule을 연속시간 동역학의 explicit Euler 이산화로 보고, ZOH 동역학의 **closed-form 해**로 대체해 이산화 오차 자체를 제거한다. dynamics matrix의 rank-1 구조 덕에 matrix exponential과 input integral이 모두 단순 갱신으로 붕괴해 파라미터·선형복잡도·chunkwise 병렬성을 보존하며, 입력에 noise/perturbation을 가하면 DeltaNet은 정확도가 붕괴하는 반면 EFLA는 강건하게 유지된다 [a][arXiv:2512.12602]. 다만 EFLA의 exactness는 retrieval 정확 저장이 아니라 discretization-error 제거이므로 HOLA와 혼동하면 안 된다(skeptic) [b].

### (i-c) 길이에 따라 자라는 memory — growing memory

Memory Caching(MC)은 sequence를 segment로 나눠 각 segment의 압축된 memory state를 checkpoint로 caching, 각 토큰이 자신의 online memory와 과거 cached memory들을 함께 참조해 전체 이력의 압축 정보에 직접 접근한다. 이로써 유효 용량이 길이에 따라 자라고, RNN의  $O(L)$ 과 Transformer의  $O(L^2)$  사이를 subquadratic으로 보간한다 — Linear Attention·deep memory 모듈 **Titans**·Sliding Window LA·Deep LA에서 검증됐다 [a][arXiv:2602.24281]. (cached checkpoint 수  $N$ 에 대해 대략  $O(N \cdot L)$ 로 비용을 조절한다는 해석은 메커니즘상 자연스러운 재서술이다 [b].) KV-Means(Key-Value Means)는 block-recurrence로 fixed 또는 growing state를 모두 수용하며, growable cache에서 sublinear state 성장·subquadratic prefill로 long-context 경쟁력을 유지하되 custom kernel 없이 chunk 병렬 학습·prefill이 가능하다 [a][arXiv:2605.09877]. 두 방식 모두 “고정 용량”이라는 전제를 완화해, 필요할 때만 용량을 지불한다.

### (i-d) 덮어쓰기 간섭을 줄이는 write — erase-write 분리

단일 delta-rule 갱신은 “key 쪽에서 얼마나 지울까”와 “value 쪽에서 얼마나 쓸까”를 하나의 scalar gate로 뭉뚱그려, 낡은 association이 남아 이후 read/write를 오염시킨다. Gated DeltaNet-2는 **channel-wise erase gate  $b_t$ (key 축)**와 **channel-wise write gate  $w_t$ (value 축)**를 분리해 fixed-state 재귀의 최대 압력점인 간섭을 직접 겨냥한다(두 gate가 같은 scalar로 붕괴하면 KDA, 여기에 decay까지 붕괴하면 Gated DeltaNet으로 환원된다) [a][arXiv:2605.22791]. Erase-then-Delta Attention(EDA)은 독립적으로 고른 erase 주소(learned erase direction)에서 stale 내용을 제거한 뒤 현재 write 주소에서 delta 보정을 수행한다 [a][arXiv:2606.26560]. erase 주소를 write와 떼어내면 다른 위치의 낡은 정보를 정밀 추출할 수 있어, 같은 용량으로 더 깨끗한 memory를 유지하는 것이 accuracy 보존의 메커니즘이다 [b].

### (i-e) feature map으로 용량 자체를 키움 — 고용량 feature map

matrix-valued memory는 선형독립 key-value를 대략  $O(d_k)$ 쌍만 저장한다. Atlas는 key/query에 **polynomial(고차) feature map**을 씌워 용량을  $O(d_k \cdot d_v)$  규모로 끌어올리고, Omega rule로 마지막 토큰이 아닌 sliding window의  $c$ 개 토큰을 함께 최적화(문맥 기억)하며, Muon으로 2차 정보를 병렬 활용한다 [a][arXiv:2505.23735]. 실무적으로 고차 다항을 그대로 펼치면 차원이 폭발하므로 symmetric-power(sympow) 형태로 축약하는 것이 자연스러운 실현이다 [c]. 고차 feature 공간은 직교 방향이 많아 collision이 줄어 associative 용량이 오르는 것이 원리다 [b].

### (i-f) 저장을 똑똑하게 — 압축

Lattice는 K-V의 low-rank 구조를 이용해 cache를 고정 개수의 memory slot으로 압축하되, 각 슬롯을 **현재 state에 직교인 성분**으로만 갱신하는 orthogonal update를 쓴다 — 새롭고 중복 아닌 정보만 흡수하므로 간섭이 억제되며, 압축은 online 최적화 문제로 정식화되어 single-GD-step 갱신 규칙(state-input-dependent gating)으로 유도된다 [a][arXiv:2504.05646]. Trellis는 KV cache를 fixed-size memory로 대체하고, forget gate를 가진 two-pass online-GD 압축(1-pass encode → 2-pass forget-gate refine)을 test-time에 학습해 LM·commonsense·recall·time-series에서 강한 baseline을 능가한다 [a][arXiv:2512.23852]. LoLA는 학습 불필요한 증강으로, KV를 sliding-window(최근)·sparse global cache(외우기 어려운 것)·recurrent state(일반)로 분배하고, hidden state의 선형 associative map과 **불일치하는 KV(self-recall error가 큰 것)만 골라 sparse global cache에 정확 저장**한다 — passkey 정확도를 0.6%→97.4%로, 4K 문맥에서 Llama-3.1-8B 대비

4.6배 작은 cache로 끌어올린다 [a][arXiv:2505.23666]. 충돌하는 소수만 정확 저장하는 것이 값싸게 exact recall을 복원하는 이유다 [b].

## (ii) physical device 개선 — footprint·bandwidth·별도 state 제거

여기서는 memory의 물리적 실체를 손본다. TTQ는 prompt별 online calibration으로 activation-aware **test-time quantization**을 수행해, offline calibration 데이터 없이 domain shift에 적응하면서 footprint를 줄이고 추론을 가속한다 [a][arXiv:2603.19296]. 다만 TTQ는 일반 LLM 가중치·activation 양자화 기법으로 TTT/Titans 계열의 fast-weight state를 직접 다루지는 않으며(“test-time”이라는 이름을 공유할 뿐), TTT state를 on-the-fly 양자화해 점유를 낮추는 것은 메커니즘상 가능한 확장에 불과하다(skeptic) [c]. KVBuffer는 linear-attention decoding에서 매 스텝 큰 recurrent state를 HBM에서 통째로 읽고 쓰는 **memory-bandwidth(IO) 병목**을 겨냥해, 최근 K,V를 buffer에 모아 state 갱신을 지연·배치하는 chunkwise 계산·speculative decoding의 parallel verification·짧은 문맥의 직접 attention을 지원한다 — SGLang(Qwen3-Next/GDN) 상에서 decoding 지연 최대 45.17% 감소, draft 4개 검증 speculative decoding에서 최대 동시요청 5배 [a][arXiv:2605.19049]. state 갱신을 지연·배치해 HBM 왕복을 상각하는 것이 원리다 [b]. In-Place TTT는 아예 별도 memory 모듈을 없앤다 — MLP block의 마지막 projection 행렬을 adaptable fast-weight로 재사용하고, generic reconstruction 대신 next-token-prediction에 정렬된 목적함수를 써 4B 모델을 128k 문맥까지 끌어올린다(ICLR 2026 oral) [a][arXiv:2604.06169]. 기존 가중치를 재활용하므로 state의 추가 메모리 오버헤드가 0이라는 점이 핵심이다 [b].

**정리하면**, (i)은 용량·간섭·저장이라는 논리적 병목을, sparsity(SSE/SDM)·exact 병치(HOLA/LoLA)·성장(MC/KV-Means)·erase-write 분리(GDN-2/EDA)·feature map 승격(Atlas)·직교·게이트 압축(Lattice/Trellis)으로 각각 공략하고, (ii)은 같은 알고리즘의 물리 실험을 quantization(TTQ)·bandwidth 상각(KVBuffer)·state 제거(In-Place TTT)로 최적화한다. 이들은 서로 다른 층위의 해법이라 상호 배타적이지 않고 결합 가능하다 [b].

**Part IV 요약.** memory device는 세 병목이 만나는 물리적 결절점 — **용량·간섭·대역폭**. 개선은 ‘더 크게(expandable/sparse)·더 정확히(exact memory)·더 싸게(quantize/in-place)’ 저장하는 세 축이며, 각 축이 특정 병목을 직접 겨눈다.

## 11. 종합 — 세 병목 × 해법 × memory device 지도

세 병목·해법·memory device를 한 장에 겹치면 다음과 같다.

| 병목           | 근본 원인                                                                | 대표 해법(무엇을)                                                                          | 왜 working                       | memory device 연결                         |
|--------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------|------------------------------------------|
| ① model-size | 문맥을 파라미터가 아니라 <b>유한 메모리 용량</b> 으로 저장( $d^2$ 에 $\sim d$ ) [b]         | distill/linearize로 규모 상속 (MOHAWK·LoLCATS·figure)<br>용량 확장(Atlas), 성장(Sleep·HOPE)    | 규모는 남의 것을 물려받고, 용량은 움           | <b>용량 상한</b> 이 병목의 진원                    |
| ② training   | inner-loop의 <b>순차 의존</b> → 작은 chunk면 FLOPs util<5% [a]               | 큰 chunk(LaCT), chunkwise-parallel(DeltaNet·TN)<br>비선형 recurrence 병렬화 (DEER·ParaRNN) | 순차성을 chunk 경계로 밀거나 병렬 형태로 접음    | <b>state footprint</b> 가 chunk·batch를 제약 |
| ③ serving    | request마다 <b>mutable state 소유</b> → batching·prefix-cache 불변식 붕괴 [a] | RW-TTT(owner 태깅 배치), hybrid prefix caching(Marconi·HYPIC), In-Place·IO-aware        | 재사용·배치 프리미티브를 recurrent에 맞게 재설계 | <b>대역폭·footprint</b> 가 decode를 bound     |

관통하는 한 줄: 모든 효율 이득은 결국 **memory device**의 유한한 용량·대역폭과 거래한다. 표현력을 위해 state를 키우면 training FLOPs·serving footprint가 오르고, 이를 줄이려 압축·양자화하면 용량·정확도가 깎인다. 프론티어는 “세 가지를 동시에” 하는 것 — LaCT(큰 chunk+큰 state), In-Place TTT(state 없이 기존 가중치 재활용), RW-TTT(요청별 state 배치 서빙)가 각 꼭짓점을 민다. 단, 이들은 서로 다른 갈래이며 아직 하나로 합쳐지지 않았다.

## 12. 열린 질문

1. **큰 chunk vs 세밀한 적응.** LaCT의 2K~1M chunk는 util을 살리지만 chunk 내부의 순차 의존을 지연시킨다. 정확도 손실 없이 둘을 동시에 얻는 chunk 스케줄의 이론적 상한은? [c]
2. **TTT  $\approx$  linear attention 등가의 함의.** 복잡한 다층-MLP·momentum TTT조차 학습된 linear attention operator로 등가 재작성된다면 [arXiv:2602.21204], “test-time 학습”의 표현력 우위는 실제로 어디서 오는가 — 진짜 meta-learning인가, 아니면 history-dependent 혼합인가? [c]
3. **용량 벽을 정말 넘는가.**  $d^2$ 에  $\sim d$ 라는 associative-memory 용량 한계를 sparse/expandable state가 상수배 개선인지, 스케일링 지수 자체를 바꾸는지. [c]
4. **request-owned state 서빙의 스케일.** RW-TTT가 수천 동시 요청에서도 성립하나 — 요청별 fast weight의 메모리 폭발과 phase 호환 배치의 한계. [b]
5. **hybrid recurrent state의 표준 캐시 프리미티브.** Marconi·HYPIC·Sparse Prefix가 제각각인데, KV cache에 준하는 표준이 나올 수 있나? [c]
6. **scratch 대규모 학습의 길.** model-size 병목이 distillation으로만 실용적으로 풀린다면, TTT/Titans를 처음부터 초대형으로 pre-train하는 경로는 닫힌 것인가? [c]
7. **재현성.** Titans Revisited[arXiv:2510.09551]의 지적처럼, neural memory의 실이득이 어느 규모·태스크에서 baseline을 확실히 넘는지 아직 합의가 없다. [a]
8. **물리-알고리즘 co-design.** memory device의 물리(HBM 용량·대역폭)와 알고리즘(state 용량·chunk)을 함께 모델링하는 비용 프레임(HATIR류)으로 무엇을 예측할 수 있나? [c]

## References

- [2006.16236] Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention — <https://arxiv.org/abs/2006.16236>
- [2008.02217] Hopfield Networks is All You Need — <https://arxiv.org/abs/2008.02217>
- [2102.11174] Linear Transformers Are Secretly Fast Weight Programmers — <https://arxiv.org/abs/2102.11174>
- [2307.08621] Retentive Network: A Successor to Transformer for Large Language Models — <https://arxiv.org/abs/2307.08621>
- [2309.06180] Efficient Memory Management for LLM Serving with PagedAttention (vLLM) — <https://arxiv.org/abs/2309.06180>
- [2309.12252] DEER: Parallelizing non-linear sequential models over the sequence length — <https://arxiv.org/abs/2309.12252>
- [2311.18677] Splitwise: Efficient Generative LLM Inference Using Phase Splitting — <https://arxiv.org/abs/2311.18677>
- [2401.09670] DistServe: Disaggregating Prefill and Decoding — <https://arxiv.org/abs/2401.09670>
- [2403.02310] Sarathi-Serve: Chunked Prefills — <https://arxiv.org/abs/2403.02310>
- [2406.06484] Parallelizing Linear Transformers with the Delta Rule over Sequence Length — <https://arxiv.org/abs/2406.06484>
- [2407.04620] Learning to (Learn at Test Time): RNNs with Expressive Hidden States (TTT-Linear/MLP) — <https://arxiv.org/abs/2407.04620>
- [2407.19115] Towards Scalable and Stable Parallelization of Nonlinear RNNs (quasi-DEER / ELK) — <https://arxiv.org/abs/2407.19115>
- [2408.10189] Transformers to SSMs: Distilling Quadratic Knowledge to Subquadratic Models (MOHAWK/Phi-Mamba) — <https://arxiv.org/abs/2408.10189>
- [2410.10254] LoLCATs: On Low-Rank Linearizing of Large Language Models — <https://arxiv.org/abs/2410.10254>
- [2411.19379] Marconi: Prefix Caching for the Era of Hybrid LLMs — <https://arxiv.org/abs/2411.19379>
- [2501.00663] Titans: Learning to Memorize at Test Time — <https://arxiv.org/abs/2501.00663>

- [2501.12352] Test-time regression: a unifying framework for designing sequence models with associative memory — <https://arxiv.org/abs/2501.12352>
- [2502.14458] Llama: Scaling Distilled Recurrent Models for Efficient Language Processing — <https://arxiv.org/abs/2502.14458>
- [2503.01496] Liger: Linearizing Large Language Models to Gated Recurrent Structures — <https://arxiv.org/abs/2503.01496>
- [2503.03269] Conformal Transformations for Symmetric Power Transformers — <https://arxiv.org/abs/2503.03269>
- [2503.14376] Tiled Flash Linear Attention: More Efficient Linear RNN and xLSTM Kernels — <https://arxiv.org/abs/2503.14376>
- [2504.03624] Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models — <https://arxiv.org/abs/2504.03624>
- [2504.05646] Lattice: Learning to Efficiently Compress the Memory — <https://arxiv.org/abs/2504.05646>
- [2505.23666] LoLA: Low-Rank Linear Attention With Sparse Caching — <https://arxiv.org/abs/2505.23666>
- [2505.23735] ATLAS: Learning to Optimally Memorize the Context at Test Time — <https://arxiv.org/abs/2505.23735>
- [2505.23884] Test-Time Training Done Right (LaCT) — <https://arxiv.org/abs/2505.23884>
- [2506.02475] Comba: Improving Bilinear RNNs with Closed-loop Control — <https://arxiv.org/abs/2506.02475>
- [2507.06457] A Systematic Analysis of Hybrid Linear Attention — <https://arxiv.org/abs/2507.06457>
- [2507.16577] Scaling Linear Attention with Sparse State Expansion — <https://arxiv.org/abs/2507.16577>
- [2508.14444] NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model — <https://arxiv.org/abs/2508.14444>
- [2508.16817] Predictability Enables Parallelization of Nonlinear State Space Models — <https://arxiv.org/abs/2508.16817>
- [2509.26030] Muon Outperforms Adam in Tail-End Associative Memory Learning — <https://arxiv.org/abs/2509.26030>
- [2510.09551] Titans Revisited: A Lightweight Reimplementation and Critical Analysis of a Test-Time Memory Model — <https://arxiv.org/abs/2510.09551>
- [2510.21450] ParaRNN: Unlocking Parallel Training of Nonlinear RNNs for Large Language Models — <https://arxiv.org/abs/2510.21450>
- [2510.26692] Kimi Linear: An Expressive, Efficient Attention Architecture — <https://arxiv.org/abs/2510.26692>
- [2510.26912] Understanding and Enhancing Mamba-Transformer Hybrids for Memory Recall and Language Modeling — <https://arxiv.org/abs/2510.26912>
- [2511.07343] TNT: Improving Chunkwise Training for Test-Time Memorization — <https://arxiv.org/abs/2511.07343>
- [2512.12602] Error-Free Linear Attention (EFLA): Exact Solution from Continuous-Time Dynamics — <https://arxiv.org/abs/2512.12602>
- [2512.23852] Trellis: Learning to Compress Key-Value Memory in Attention Models — <https://arxiv.org/abs/2512.23852>
- [2512.24695] Nested Learning: The Illusion of Deep Learning Architectures (HOPE) — <https://arxiv.org/abs/2512.24695>
- [2602.21204] Test-Time Training with KV Binding Is Secretly Linear Attention — <https://arxiv.org/abs/2602.21204>
- [2602.24281] Memory Caching: RNNs with Growing Memory — <https://arxiv.org/abs/2602.24281>
- [2603.19296] TTQ: Activation-Aware Test-Time Quantization to Accelerate LLM Inference On The Fly — <https://arxiv.org/abs/2603.19296>
- [2604.06169] In-Place Test-Time Training — <https://arxiv.org/abs/2604.06169>
- [2605.05066] The Impossibility Triangle of Long-Context Modeling — <https://arxiv.org/abs/2605.05066>
- [2605.05219] Sparse Prefix Caching for Hybrid and Recurrent LLM Serving — <https://arxiv.org/abs/2605.05219>
- [2605.09877] Key-Value Means: Transformers with Expandable Block-Recurrent Compressed Memory — <https://arxiv.org/abs/2605.09877>
- [2605.10795] (제목 확인 필요) — <https://arxiv.org/abs/2605.10795>
- [2605.12049] Scaling Laws and Tradeoffs in Recurrent Networks of Expressive Neurons — <https://arxiv.org/abs/2605.12049>
- [2605.19049] KVBuffer: IO-aware Serving for Linear Attention — <https://arxiv.org/abs/2605.19049>
- [2605.22791] Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention — <https://arxiv.org/abs/2605.22791>
- [2605.28053] RW-TTT: Batched Serving for Request-Owned Test-Time Training State — <https://arxiv.org/abs/2605.28053>

<https://arxiv.org/abs/2605.28053>

- [2606.03979] Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories — <https://arxiv.org/abs/2606.03979>
- [2606.08635] (제목 확인 필요) — <https://arxiv.org/abs/2606.08635>
- [2606.26560] Erase-then-Delta Attention: Decoupling Erase and Write Addresses in Delta-Rule Linear Attention — <https://arxiv.org/abs/2606.26560>
- [2607.01299] HYPIC: Accelerating Hybrid-Attention LLM Serving with Position-Independent Caching — <https://arxiv.org/abs/2607.01299>
- [2607.02303] A Hippocampus for Linear Attention: An Exact Memory for What the Recurrent State Forgets — <https://arxiv.org/abs/2607.02303>
- [2607.07386] Sparse Delta Memory: Scaling the State of Linear RNNs through Sparsity — <https://arxiv.org/abs/2607.07386>

## 부록 A — 섹션별 검증 노트 (사실검증 요약)

각 섹션은 집필 후 별도 에이전트가 1차 문헌과 대조해 수치·인용·근거등급을 교정했다. 주요 교정 요약:

- **1. TTT 집중 조명 — test-time에 학습되는 메모리 시스템** — 7개 arXiv id 전부 실재·제목·저자 확인 (WebSearch/원문 PDF). 주요 교정: (1) LaCT의 >70% FLOPs 이용률은 **H100**이 맞음 — 초안의 “A100”은 오류(A100은 원 TTT 논문의 TensorCore 설명에 등장, 두 논문 혼동)로 교정. “최대 70%”→“peak FLOPs의 70%를 넘는 수준”. (2) TTT dual form “ $>5\times$  faster”는 논문의 **JAX 구현 기준**(primal 대비)이므로 “TPU에서”→“논문의 JAX 구현 기준”으로 정정, [a] 유지. dual form 문단에 TensorCore가 노는 이유(순차 벡터갱신)를 원문 근거로 보강. (3) 2602.21204는 “up to **4.0**  $\times$  **inference throughput** on architectures with complex multi-layer MLP inner loops”이며 momentum·다층 MLP 등가재작성은 Theorem 5.2/5.3로 명시 — 초안 [a] 정당, “4.0배 throughput”→“최대 4.0배 inference throughput”으로 정밀화, 등가재작성이 ‘주장’이 아니라 ‘증명’임을 반영
  - **2. Titans 집중 조명과 그 효율 전반** — 6개 arXiv id 전부 실재 확인(WebSearch). 주요 교정: (1) Atlas “Titans 대비 +80% 정확도” → 초록 원문은 “achieving +80% accuracy in 10M context length of BABILong”으로, +80은 Atlas가 10M에서 도달한 절대 정확도(~80%+)이자 Titans 장문 성능 개선. ‘+80 percentage points’로 오독될 표현을 절대치+개선으로 교정. (2) HOPE “needle-in-a-haystack에서 Titans·TTT·Mamba2 상회 보고” → 초록은 그 특정 3자 비교/NIAH 우위를 명시하지 않음 (language modeling·knowledge incorporation·few-shot·continual learning·long-context reasoning의 promising results만 명시). 미확인 비교 주장 제거하고 초록 문구로 대체. (3) Sleep: 초록의 2단계는 Memory Consolidation (Knowledge Seeding=upward distill)과 Dreaming(RL 합성 커리큘럼)임을 명확화(초안의 ‘wake/저빈도
  - **3. 병목 ① — model-size scaling (TTT/Titans)** — 9개 arXiv id 전부 실존·주제 일치 확인(허구 인용 없음). ATLAS(2505.23735) 용량 주장 3건은 원문 Prop1(matrix  $O(d_k)$ , sub-linear)·Thm1(MLP  $\geq 2$  층 최소  $O(d_k d_v)$ , subquadratic 상한)·Prop2(poly  $\phi_p \rightarrow O(d_k^p)$ )와 정확히 일치 → [a] 유지, 단  $O(\text{binom}(d_k+p,p))$  표기는 원문이  $O(d_k^p)$ 라서 단항식 개수 유래로 재서술하고 [b] 강등. Omega Rule 정의(sliding window c) 확인. 교정: (1) Hopfield 0.14d→0.138N(N=뉴런 수) 정밀화; (2) factual-recall 문턱을 임의의 ‘ $d^2 \approx n \log n$ ’에서 원문 정확식  $p_c \log p_c / d^2 = 1/2$  ( $d^2 \approx 2 p_c \log p_c$ )로 교정; (3) Titans Revisited MRR 수치 실검증 성공—원문 값 Titans·attn only 0.3418→+memory 0.4371, BERT4Rec 0.4451로 명기, 기본 chunk 32, ‘larger chunk better but costlier’와 ‘memory가 chunking 손실 완화
  - **4. 병목 ② — training parallel/batch scaling (TTT/Titans)** — (1) [교정·사실오류] LaCT(arXiv:2505.23884)의 70% utilization 하드웨어는 H100이 아니라 **A100**. 또한 논문 표현은 “peak FLOPs 70% 이상”이 아니라 “GPU utilization 최대 70%”이므로 “70% 이상”→“최대 70%”로 하향. <5% FLOPs util, 16/64토큰 갱신, 2K~1M chunk, nonlinear state 최대 40%는 모두 논문 명시 [a] 확인.
- (2) [교정·과장] Gonzalez·Kozachkov(arXiv:2508.16817)는 “예측가능=한 자릿수 빠름 / chaotic=한 자릿수 느림”이라고 **말하지 않는다**. 실제 주장: 예측가능 시스템은  $O((\log T)^2)$  병렬시간, chaotic/예측불가능 조건수가 나빠 parallel 수렴이 “너무 느려 무용”. 초안의 order-of-magnitude 수치 삭제, [a] 유지하되 문구를 논문 그대로 교정.
- (3) [확인] TNT(arXiv:2511.07343): “가장 정확한 baseline 대비 최대 17배”, “주기적 local memory 리셋으로 context parallelization”, chun

- **5. 병목 ③ — serving batch inference + Prefill/Decode(P/D) scaling** — 11개 arXiv id 전부 WebSearch로 실재 확인(허구 없음). 주요 교정: (1) HYPIC 수치 오류 교정 — 초안 “TTFT 2.0×”는 논문의 TTFT/throughput 수치를 뒤섞은 것. 원문 = TTFT 최대 2.45× 감소 + peak throughput 최대 2.0× 향상(4 hybrid 모델/5 workload, 동일 SLO). (2) DistServe — 초안 “쿼리당 비용 7.4× 감소[a]”는 논문 미명시 재해석. 원 주장 = 지연 제약 하 7.4× 더 많은 request 또는 12.6× 더 빠른 SLO로 교정. (3) Marconi — “latency 71.1% 감소”를 원문대로 “TTFT 71.1%(약 617ms) 감소”로 명확화; token hit rate 34.4×는 정확. 확인 완료(변경 없음): In-Place TTT의 fast/slow weight 구조[a], RW-TTT 274.61 tok/s·9.31×·3.44×[a] 및 RULER 보존, Splitwise 2.35×[a], Kimi Linear KV 75%·decode 6×@1M[a], Sparse Prefix Caching exact O(NM) DP[a],
- **6. 해법 ① — model-size scaling 해결** — 주요 교정 — (1) [Sleep 2606.03979] 원 초안의 “sparse MoE + router”는 논문에 없음(허구). 실제로는 parameter expansion + Knowledge Seeding(작은-self→큰 network 상향 distill) + Dreaming(RL 합성 커리큘럼)로 교정, MoE 메커니즘 문장 삭제. (2) [LoLCATs 2410.10254] “405B에서 +38.3점 회복”은 abstract에 없음(허구) → 삭제. 대신 검증된 “8B·7B급 +20점 이상” 추가; 77.8%/78.1% gap-closing은 사실 확인. (3) [MOHAWK 2408.10189] “from-scratch 대비 1% 미만” 미명시(3B/사전학습량 기준 대략 2%대) → 논문 표현인 “33× 적은 토큰, 평균 +5%”로 교정. (4) [Liger 2503.01496] 0.02% 토큰·93% 회복 = abstract에서 사실 확인(단, Liger Attention 병용 수치임을 명시); “추가 파라미터 0”은 “추가 feature-map 모듈 없음/기존 key proj 재활용”으로 정밀화. (5) hybrid 문단: Liger Atten
- **7. 해법 ② — training parallel/batch scaling 해결** — (1) LaCT 하드웨어 utilization 70%는 H100이 아니라 A100 기준(<5%→최대 ~70%, 수십 줄 PyTorch); “H100 peak FLOPs” 표현은 근거 없어 일반 GPU로 정정. nonlinear state 최대 ~40% of params, SwiGLU-MLP fast weight는 논문 명시 [a] 확인. (2) DeltaNet 스케일은 “표준 LM 규모”가 아니라 1.3B/100B tokens로 구체화; WY 동치성의 정확도 무손실은 논문이 수치가 아닌 유도론 주장하므로 관련 문장 [a]→[b] 유지. (3) TNT 17배는 “the most accurate baseline configuration” 대비 + 정확도 동시 향상, 논문 명시 확인 [a]; skeptic 노트의 baseline-민감성은 [c]→[b]로 소폭 조정. (4) DEER: 3자리수 GPU가속·10배 학습가속·아키텍처 무관 논문 명시 [a]; 단 “cubic in state size + 수치 불안정 한계를 DEER가 명시”라는 원문 귀속은 부정확 — 이 한계를 정면으로 다룬 것은 후속 quasi-DEER/ELK(arXiv:2407.19115)이므로 실존
- **8. 해법 ③ — serving P/D batch inference 해결** — (1) 모든 arXiv id·인용 실재 확인 — RW-TTT(2605.28053, 274.61 tok/s·9.31×·3.44× 정확), Marconi(2411.19379, MLSys’25, 34.4×·71.1%/617ms 정확), Sparse Prefix Caching(2605.05219, O(NM) DP·exact suffix recompute 확인, 정확성 보존은 논문 명시라 [b]→[a]로 상향), KVBuffer(2605.19049), In-Place TTT(2604.06169, ICLR2026 Oral), Nemotron-H(2504.03624, 3× 확인), Nemotron Nano 2(2508.14444), Kimi Linear(2510.26692, 3:1 KDA:MLA·75%·6× 확인), SpectrumKV(2606.08635), vLLM 블로그(2026-04-21) 모두 실재. 허구 인용 없음. (2) 최대 교정 — HYPIC(2607.01299) 수치가 초록과 전부 불일치했음: 초안 “TTFT 2.45×/peak throughput 2.0×/정확도 3.3 point 이내/8-worker cold-prefill 6.1×” → 실제 초록’ 평
- **9. memory device 집중 조명 ① — 한계** — 인용된 arXiv id 7개(2008.02217 Hopfield, 2505.23735 Atlas, 2501.00663 Titans, 2503.03269 Conformal-sympow, 2605.05066 Impossibility Triangle, 2510.26912 Mamba-Transformer hybrid, 2605.19049 KVBuffer) 모두 실재하며 귀속 정확 — 허구 인용 없음. 교정: (1) 메커니즘 오류 — 최단순 신경메모리  $M+ = vk^T$ 는 delta rule이 아니라 outer-product(Hebbian) rule; delta rule  $M+ = (v - Mk)k^T$ 은 TTT/DeltaNet의 error-correcting 변형임을 분리·명시. (2) Impossibility Triangle에 논문의 실제 정량 하한  $O(\text{poly}(d)/\log V) \rightarrow o(T)$ 와 Online Sequence Processor 틀 보강(근거 [a] 유지, 정확). (3) KVBuffer 5x 수치에 원문 조건 “draft token 4개 검증 시” 명시; 45.17% latency 감소·“state가 per-
- **10. memory device 집중 조명 ② — 개선으로 해결되는 해법** — 15개 arXiv id 전부 WebSearch/WebFetch로 실재 확인, 허구 인용 없음. 주요 교정: (1) HOLA 수치(340M/15B SlimPajama, 27.32→22.92, Transformer++ 26.88, 32k=16x RULER NIAH) 원문 대조 완료 — 모두 정확, cache 선정 신호가 “intrinsic surprise”가 아니라 write strength  $\beta \times \text{residual} \parallel e \parallel$  (학습된 eviction 모듈 없음)임을 원문 표현으로 교정, 정식 명칭 ‘A Hippocampus for Linear Attention’로 보정. (2) GDN-2 메커니즘 원문 확인 — channel-wise erase gate  $b_t(\text{key})/\text{write gate } w_t(\text{value})$  분리 정확, 기존 초안의 “단일 delta 주소” 표현을 “단일 scalar gate”로 정밀화, KDA/GDN 환원 관계 추가. (3) MC: Titans 검증 확인, 그러나 원문은  $O(L) \sim O(L^2)$  subquadratic 보간이라 명시 — 초안의  $O(NL)$ 은 [a]→[b] 재서술로 강등. (4) EFLA: 원문의 강건성 근거는 pixel dropout/noise